José Valente de Oliveira e Fernando Lobo

Introdução à

Programação de Computadores em Java

Introdução à Programação de Computadores em Java

José Valente de Oliveira e Fernando Lobo
The Ualg Informatics Lab
Universidade do Algarve

Índice

PREÂMBULO....4

CONCEITOS FUNDAMENTAIS....5

Componentes principais de um computador....7

Linguagens de programação....8

Compiladores....9

Sistemas operativos....11

ASPECTOS DA CRIAÇÃO DE UM PROGRAMA....13

Exercícios....18

CODIFICAÇÃO DE INFORMAÇÃO....25

Unidades de informação: bits, bytes, kilobytes, ...25

Números inteiros....26

Sistemas de numeração....27

Gamas de variação para inteiros....29

Caracteres....30

Noção de tipo de dados....31

Exercícios....32

PRIMEIROS PROGRAMAS....33

Explicação do primeiro programa....34

Segundo programa....35

Tipos primitivos de dados....36

Programa para somar 2 números inteiros....37

Explicação do programa....37

Exercícios....40

Primeiros aspectos sobre a documentação de código....40

Expressões....43

Exercícios....46

ESTRUTURAS DE CONTROLO DE FLUXO 48

A instrução de selecção simples if 49

A instrução de selecção em alternativa if else 51

Exercícios 53

O operador condicional 54

A instrução de selecção múltipla switch 55

Exercícios 57

Iteradores 59
A instrução while 60
A instrução for 64
A instrução do-while 67
Instruções de salto incondicional 69

Apêndice A – O formalismo EBNF 72

MODULARIDADE DE FUNÇÕES 74

Definindo funções 78
Parâmetros e tipo de retorno 79

Noção de variável local 80

Noção de alcance 82

Primeira referência à Qualidade em funções 82

Exercícios 83

Primeiras noções de recursividade 83

Aplicações da recursividade 85
Definição sintáctica de instruções 85
Conceito de antepassado 86
Conjuntos indutivos 87

Exercícios 87

TABELAS 89

Declaração, definição e acesso a elementos de uma tabela 90

Noção de referência 93

Tabelas como argumento de funções 94

Exercícios 97

Preâmbulo

Este texto constitui os apontamentos teórico-práticos para a disciplina de Programação de Computadores I das licenciaturas em Engenharia Informática e Electrotécnica da Universidade Nacional de Timor Leste, elaborados pelo primeiro autor a quando da leccionação da mesma no ano lectivo 2004/2005.

O texto, que também se encontra disponível em http://w3.ualg.pt/~jvo/timor, resulta de uma extensão e revisão dos apontamentos que o segundo autor preparou para a disciplina de Programação Imperativa (http://www.deei.fct.ualg.pt/PI_flobo/), leccionada em C e oferecida às licenciaturas em Informática, Ensino de Informática e Engenharia de Sistemas e Informática da Universidade do Algarve.

Capítulo

1

Conceitos fundamentais

Introduzem-se noções fundamentais como tipos de computadores, componentes de um computador; linguagem de programação; compiladores e interpretadores; sistemas operativos.

Distinguir entre diferentes tipos de computadores nem sempre é fácil. Habitualmente usam-se nomes como os seguintes que nos dão alguma informação sobre as capacidades do computador e o fim a que se destinam.

Super-computadores (***main frames***): são computadores topo de gama e normalmente só existem em grandes laboratórios de investigação ou em grandes empresas.



Computadores pessoais de secretária (*desktop computers*): também chamados PCs (*personal computers*) e são o tipo de computador que temos na sala de aulas.



Computadores portáteis (*laptops* ou *notebooks*): Têm características semelhantes aos computadores de secretária, mas são mais pequenos e leves.



Computadores de bolso (PDA - Personal Digital Assistants): São ainda mais pequenos; cabem na palma da mão.



Mas o que é um computador? Numa frase, a resposta pode ser: uma máquina programável, de aplicação genérica.

O comando de uma televisão também é programável… O que é que distingue então um computador de uma televisão?

Componentes principais de um computador

A parte física do computador (muito vezes chamada *hardware*) é constituída por uma unidade central e por um conjunto de acessórios ou periféricos que tipicamente incluem o teclado, o rato, o monitor (ou ecrã) e cada vez mais um microfone, um conjunto de colunas e mesmo uma (pequena) câmara.

A unidade central tem dois componentes essenciais: o processador e a memória. O processador é o componente que manipula a informação, i.e. "faz as contas". Naturalmente que a memória serve para armazenar informação. Existem essencialmente dois tipos de memória. A memória de acesso rápido (RAM – *Random Acess Memory* e ROM – *Read Only Memory*) e a memória de massa (disco). A memória RAM é usada pelo computador para armazenamento temporário de dados. Uma vez desligado o computador, o conteúdo da memória RAM desaparece. Ao contrário da memória RAM, a informação armazenada na memória do disco permanece mesmo quando desligamos o computador. Por esta razão, a memória RAM diz-se volátil.

Os computadores podem manipular diversos tipos de informação, incluindo:

- números
- texto (ex: nomes de pessoas, moradas)
- imagens (ex: desenhos, gráficos, fotografias)
- vídeos (ex: efeitos especiais nos filmes do Star Wars)
- som

Para que o computador manipule a informação é necessário que nós lhe digamos exactamente aquilo que ele deve fazer. Usamos para isso um conjunto de *instruções*. Esse conjunto de instruções é uma espécie de "receita de cozinha" que pode ser armazenado na memória do computador. A esta "receita" chamaremos *programa*. Depois de armazenado, o programa pode ser executado. Isso desencadeia as seguintes acções:

1. Um conjunto de informação ou dados, chamados *dados de entrada* são introduzidos no computador (por exemplo, através do teclado ou rato)

2. Os dados de entrada são manipulados para produzir determinados resultados chamados *dados de saída*.
3. Os dados de saída (e possivelmente os dados de entrada) são guardados num ficheiro em disco ou mostrados no ecrã.

Estes três passos podem ser repetidos muitas vezes. É de salientar também que cada um destes passos tanto pode ser uma tarefa curta e simples como uma tarefa longa e complicada. Os dados de entrada são por vezes chamados *input* e os dados de saída *output*.

Exemplo: pretende-se programar um computador para calcular a área de um círculo usando a fórmula `a = pi*r²`, introduzindo um valor numérico para o raio `r` como dado de entrada.
Para fazer o programa pretendido, temos de dizer ao computador para executar as seguintes tarefas:

1. Ler o valor numérico para o raio do círculo e guardar esse valor na memória do computador.
2. Calcular a área utilizando a fórmula e guardar o resultado em memória.
3. Mostrar o resultado no ecrã.

Cada uma destas tarefas pode necessitar de uma ou mais instruções num programa de computador.

Linguagens de programação

As tarefas 1, 2, 3, especificadas acima foram escritas em Português. Infelizmente, os computadores não entendem linguagens naturais como o Português, apenas entendem linguagens de computador. Do mesmo modo que existem várias linguagens naturais (Português, Tétum, Inglês, etc), também existem várias linguagens de programação de computadores (C, C++, Java, etc). Nesta disciplina vamos aprender os fundamentos da programação de computadores usando a linguagem Java. Os princípios fundamentais do Java são comuns a uma boa parte das linguagens de programação. O mesmo se

passa em relação às linguagens naturais. Quem sabe Português e Espanhol, terá menos dificuldades em aprender Italiano, do que quem não sabe. Da mesma forma, quem tem conhecimentos sólidos de Java, terá também menos dificuldades em aprender outra linguagem de programação.

Compiladores

Os computadores não entendem directamente uma linguagem de programação como o Java, apenas entendem a chamada *linguagem máquina*. Esta linguagem varia de computador para computador dependendo do tipo de processador. É possível fazer programas directamente em linguagem máquina mas quase ninguém o faz porque isso é uma tarefa um pouco morosa e complicada. Um programa em linguagem máquina é incompreensível e tem mais ou menos o seguinte aspecto:

```
00000010011101110111
00000010010110000101
00000101111011010100
```

A linguagem máquina é constituída por 0s e 1s porque os computadores são máquinas electrónicas. Como tal, aquilo que eles entendem directamente são sinais electrónicos. Esses sinais electrónicos têm dois estados: ligado e desligado, que são aqui representados por 1 e 0, respectivamente. O facto de o computador só trabalhar com dois símbolos não limita aquilo que pode fazer. As pessoas também só trabalham com 23 letras mas o número de palavras e de frases que conseguimos escrever com essas 23 letras é ilimitado. As "frases" de 0s e 1s da linguagem máquina são representam instruções, números, letras, e variáveis. Por exemplo, o conjunto de 0s e 1s que vimos à pouco são poderão representar o seguinte.

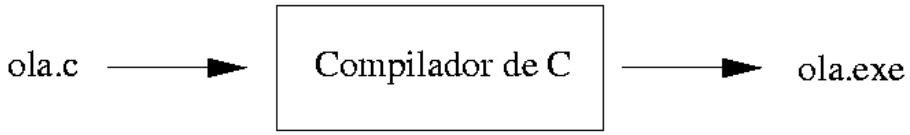
```
LOAD  I
ADD   5
STORE K
```

Estas 3 instruções são uma variante simbólica das instruções em linguagem máquina. A variante simbólica chama-se *linguagem assembly* e é um pouco mais perceptível do que uma sequência de 0s e 1s. Apesar de tudo, não é tão clara como:

```
k = i + 5
```

Esta última instrução já é parecida com a notação matemática a que estamos habituados. As instruções de linguagens como o C, C++ ou Java, têm este aspecto. Estas linguagens chamam-se *linguagens de alto nível* porque estão a um nível relativamente distante dos detalhes internos do computador. Ao contrário destas, a linguagem máquina e a sua variante simbólica (o assembly) chamam-se *linguagens de baixo nível*.

É muito mais simples programar utilizando uma linguagem de programação de alto nível. Assim, o que se costuma fazer é utilizar um programa especial que traduz automaticamente programas escritos em linguagens como o C para a linguagem máquina que o computador entende. Esses tradutores chamam-se *compiladores*.



A figura acima ilustra o trabalho do compilador. O ficheiro `ola.c` é um programa escrito em linguagem C. O `ola.c` é submetido ao compilador, e este traduz o `ola.c` para o ficheiro `ola.exe` que é composto por instruções em linguagem máquina.

Se o nosso programa especial em vez de gerar código directamente para a máquina física gerar código para uma máquina virtual então toma o nome de *interpretador*. Uma máquina virtual não é mais do que um programa que simula um processador simplificado. É o que acontece no Java. Em Java o nosso programa especial chama-se javac e em vez de gerar sequência de 0s e 1s para o nosso computador físico, gera aquilo a que chama *bytecodes*, *i.e.*

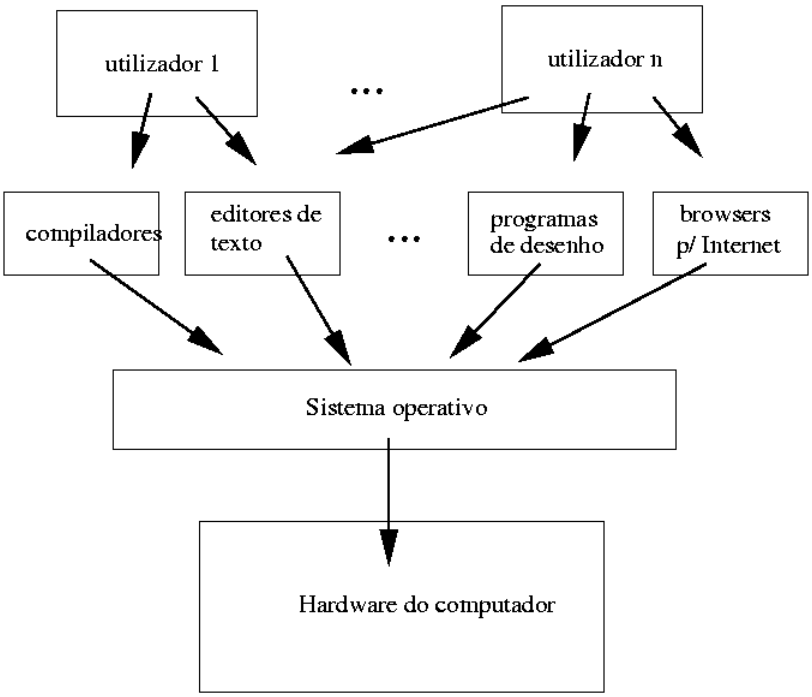
instruções para a máquina virtual JVM – Java Virtual Machine. O que é que se ganha com isto?

A linguagem máquina e o assembly não são universais, variando de processador para processador. Essa é uma das grandes vantagens de se programar numa linguagem de alto nível. Além dessa vantagem, existem outras:

- a programação é mais fácil
- os programas ficam mais legíveis
- o mesmo programa pode funcionar em computadores diferentes (basta arranjar um compilador adequado).

Sistemas operativos

Um computador só funciona se tiver um sistema operativo (actualmente, os sistemas operativos mais conhecidos e usados são o Windows e o Linux, mas existem outros). Um sistema operativo é constituído por um conjunto de programas que fazem de interface entre o hardware do computador (disco, memória, rato, teclado, unidade de processamento) e os outros programas de aplicação (compiladores, editores de texto, programas de desenho, folhas de cálculo, sistemas de bases de dados, jogos, browsers para aceder à Internet).



O sistema operativo controla e coordena a utilização do hardware pelos vários programas de aplicação. O objectivo do sistema operativo é fornecer um ambiente de trabalho para que os outros programas e utilizadores possam funcionar.

Aos programas que funcionam num computador, sejam aplicações, compiladores, interpretadores, jogos ou sistemas operativos, chama-se muitas vezes *software*.

Capítulo

II

Aspectos da criação de um programa

Apresenta-se uma primeira abordagem à resolução de problemas; Introduz-se a noção de algoritmo e algumas técnicas de especificação de algoritmos.

O que é necessário para fazer um programa de computador? Uma resposta simplista seria:

1. Um editor de texto, onde escrever as instruções do programa;
2. Um compilador para traduzir o programa numa linguagem executável pelo computador;

A questão fundamental é esta: os programas devem resolver problemas. Como resolver um problema usando um computador? Vejamos:

1. Analisar o problema, partindo-o eventualmente em problemas mais simples – a esta fase chama-se a fase de análise;
2. Desenvolver uma forma de resolver o problema, ou cada um dos vários subproblemas; à forma de resolver um problema chama-se algoritmo;
3. Traduzir o algoritmo numa linguagem de programação, *i.e.*, obter um programa;
4. Compilar e corrigir eventuais erros de compilação, tantas vezes quantas as necessárias;
5. Verificar se os resultados produzidos pelo programa correspondem ao que se pretende. Repetir uma ou mais fases, se necessário.

Um algoritmo é, portanto, um método de resolver um problema. Este problema não tem que ser necessariamente um problema do mundo dos computadores. Mais concretamente, um algoritmo é um conjunto finito de instruções precisas e ordenadas que resolve um determinado problema.

Podemos agora rever a nossa noção de programa: um programa é a tradução para uma linguagem de programação de um algoritmo, que permite a resolução de um problema.

Um algoritmo deve satisfazer o seguinte:

1. Deve terminar num número finito de passos (*finitude*);
2. Cada passo tem que ser precisamente definido (*definitude*);
3. Cada passo deve ser suficientemente simples para que possa ser resolvido "à mão" (*eficácia*);
4. Deve resolver o problema em tempo útil para que a solução tenha interesse (*eficiência*);
5. Tem 0 ou mais entradas;
6. Tem 1 ou mais saídas;

É exactamente no desenvolvimento do algoritmo onde está a maior dificuldade, mas também a fase mais divertida da resolução de problemas.

No desenvolvimento do algoritmo é fundamental identificar:

1. Estado inicial, *i.e.*, o conjunto das suas entradas;
2. A transformação, *i.e.*, os passos necessários para a partir do estado inicial chegar ao estado final;
3. O estado final, *i.e.*, as saídas desejadas

Vamos ver um exemplo de algoritmo para resolver um problema, não de programação, mas que todos conhecemos do nosso dia a dia: fazer um nescafé.

Estado inicial: água, uma colher de nescafé e uma colher de açúcar.
Estado final: um aromático nescafé
Transformação: aquecer e misturar os ingredientes.

Mas isto é muito geral, no que se refere à transformação teremos que ser mais específicos:

1. Aquecer a água até estar bem quente;
2. Misturar os ingredientes

Quem nunca fez um aromático nescafé poderá necessitar ainda de mais algumas indicações, como as seguintes:

1. Aquecer a água até estar bem quente;
2. Misturar os ingredientes
    2.1. Colocar a colher de nescafé numa chávena;
    2.2. Juntar uma gota de água;
    2.3. Bater bem o nescafé até obtermos um creme;
    2.4. Juntar o resto da água quente;
    2.5. Adicionar a colher de açúcar;
    2.6. Mexer tudo duas ou três vezes;

É importante notar que existe uma ordem certa entre cada passo. Se trocarmos o passo 2.1 com o 2.5, o resultado não seria o mesmo.

Neste caso, a verificação seria provar a bebida que acabamos de preparar e confirmar se corresponde ao desejado. Caso não corresponda, teremos que fazer algumas alterações (p.e., adicionar um pouco mais de açúcar) ou mesmo repetir todo o processo.

Note-se ainda que no desenvolvimento do nosso algoritmo decompusemos um problema maior (aquecer e misturar os ingredientes) em dois sub-problemas mais simples *aquecer a água* e *misturar os ingredientes*. O nosso estado final resultou da resolução individual destes subproblemas.

É exactamente este tipo de raciocínio que vamos fazer em programação.

Vejamos agora a resolução de problema em computador: elaborar um algoritmo que calcule a média das notas dos alunos de uma turma.

Estado inicial:	uma tabela com o nome e as notas dos alunos;
Estado final:	a média das notas;
Transformação:	Calcular o número de alunos e a soma das suas notas;
	Calcular a média dividindo a soma pelo número de alunos.

A maior parte das pessoas conseguiriam resolver este problema com a descrição acima. Infelizmente, um computador necessita de (muito) mais informação. O computador necessita que os algoritmos sejam descritos usando *passos elementares* como ler um valor, guardar um valor, adicionar dois valores. Por isso, o nosso algoritmo necessita de ser descrito, como se segue:

1. Iniciar um valor a zero, que servirá para contar o número de alunos; chamaremos a este valor simplesmente *número*;
2. Iniciar outro valor a zero, que servirá para guardar a soma das notas; chamaremos a este valor *soma*;
3. Ler a nota do primeiro aluno da tabela
4. Adicionar 1 a *número*;
5. Adicionar a nota do aluno a *soma*;
6. Ler a nota do segundo aluno da tabela
7. Adicionar 1 a *número*;
8. Adicionar a nota do aluno a *soma*;
9. …
10. Ler a nota do último aluno da tabela
11. Adicionar 1 a *número*;
12. Adicionar a nota do aluno a *soma*;
13. Dividir *soma* por *número*;
14. Guardar o resultado;
15. Escrever o resultado

Se repararmos bem os passos 3, 4 e 5, são muito semelhantes aos passos 6, 7, e 8, e assim como a muitos outros. Podemos re-escrever este mesmo algoritmo de

uma forma mais compacta, ganhando com isso, outras coisas *legibilidade, i.e.,* torna-se mais fácil para nós, perceber o que é que o algoritmo faz.

1. Iniciar um valor a zero, que servirá para contar o número de alunos; chamaremos a este valor simplesmente *número*;
2. Iniciar outro valor a zero, que servirá para guardar a soma das notas; chamaremos a este valor *soma*;
3. Repetir os passos seguintes até chegar ao último aluno
   3.1. Ler a nota do próximo aluno da tabela
   3.2. Adicionar 1 a *número*;
   3.3. Adicionar a nota do aluno a *soma*;
4. Dividir *soma* por *número*;
5. Guardar o resultado;
6. Escrever o resultado

Como vimos após a construção do algoritmo é necessário verificar se ele faz o que pretendemos. Uma das formas mais utilizada de verificação consiste em escolher um estado inicial qualquer (valor inicial das entradas), seguir à mão os passos que o algoritmo tem de seguir e verificar se o resultado gerado é igual ao que devia ser. Neste tipo de verificação devemos dar particular atenção aos valores extremos do estado inicial, *i.e.* ao maior e ao menor que as entradas podem tomar.

Se dermos como entrada imaginária do algoritmo a seguinte tabela:

João	16
Maria	10
Natércia	12

Podemos até ficar convencidos que o algoritmo funciona. E o que acontece quando a nossa entrada imaginária é uma tabela vazia? Nesse caso, o algoritmo saltaria directamente do passo 3 para o passo 4 e tentaria fazer a divisão. Mas, nesse momento, *soma* e *número* teriam ambos o valor zero. Porquê? Tentar dividir

zero por zero resultaria em erro, pelo que tem que ser evitado. Isto ilustra a utilidade de usarmos algum tipo de verificação.

O algoritmo corrigido, a que chamaremos **Calcula Média** seria:

Algoritmo Calcula Média

1. Iniciar um valor a zero, que servirá para contar o número de alunos; chamaremos a este valor simplesmente *número*;
2. Iniciar outro valor a zero, que servirá para guardar a soma das notas; chamaremos a este valor *soma*;
3. Repetir os passos seguintes até chegar ao último aluno
   3.1. Ler a nota do próximo aluno da tabela
   3.2. Adicionar 1 a *número*;
   3.3. Adicionar a nota do aluno a *soma*;
4. Se número for zero
   4.1. Escrever Tabela vazia
5. Senão
   5.1. Dividir *soma* por *número*;
   5.2. Guardar o resultado;
   5.3. Escrever o resultado

O tipo de verificação que usamos é um dos mais fáceis de utilizar e será aquele que utilizaremos ao longo da disciplina. No entanto, fica desde já o aviso que este tipo de verificação não é completamente fiável. Existem vários outros tipos de verificação mais poderosos mas não cabem numa disciplina introdutória.

Exercícios

Para cada um dos problemas seguintes desenvolver um algoritmo suficientemente detalhado que o resolva:

1. Ir da UNTL a Pante-Makasâr

2. Calcular a soma dos 10 primeiros números naturais
3. Calcula a soma dos 10 primeiros números naturais pares
4. Procurar nota de aluno em pauta;
5. Verificar se um número está entre 0 e 10
6. Ordenar por ordem crescente dois números dados;
7. Ordenar por ordem crescente três números dados;
8. Calcular o número de dígitos de um número

Especificação de algoritmos

O Português narrativo, como a maior parte das línguas naturais, apresenta algumas ambiguidades pelo que não deve ser usado na especificação de algoritmos. Á medida que vamos removendo ambiguidade ao Português, vamos obtendo subconjuntos da linguagem conhecidos por Português compacto e Português estruturado, respectivamente.

O Português estruturado é um dos métodos mais usados de especificação (ou descrição) de algoritmos. Os algoritmos descritos anteriormente utilizam Português estruturado.

No Português estruturado é um subconjunto do Português que pretende ser conciso e preciso. Para isso retiraram-se:

- adjectivos;
- advérbios;
- frases compostas ou complexas;
- todos os tempos verbais excepto imperativo ou infinitivo;
- A maioria dos sinais de pontuação.

É possível especificar qualquer algoritmo usando os seguintes tipos de expressões em Português estruturado: acções elementares, sequências, selecções e iterações.

Acções elementares. As acções elementares indicam o que de primitivo deve ser feito. São escritas como expressões imperativas ou infinitivas. Exemplo:

Ler a nota do próximo aluno da tabela

Sequências. As sequências correspondem a um encadeamento de uma ou mais acções elementares, como em:

Ler a nota do próximo aluno da tabela

Adicionar 1 a número;

Adicionar a nota do aluno a soma;

Selecções. As selecções permitem escolher exactamente uma alternativa de um conjunto de alternativas. São habitualmente representadas na forma:

Se <condição>

<acção 1>

senão

<acção 2>

como em:

Se número for zero

Escrever Tabela vazia

Senão

Dividir *soma* por *número*;

Iterações. As iterações definem uma repetição limitada de uma sequência de acções. Escrevem-se habitualmente na forma:

Enquanto <condição> faz <acção>

ou

Repetir <acção> até <condição>

como em:

Repetir os passos seguintes até chegar ao último aluno

Ler a nota do próximo aluno da tabela

Adicionar 1 a número;

Adicionar a nota do aluno a soma;

O *pseudo-código* é ainda mais restrito do que o Português estruturado e pode incluir alguns detalhes de implementação escritos mesmo com elementos da linguagem de programação que estamos a usar. O pseudo-código permite, portanto, descrever um algoritmo de forma concisa sem nos preocuparmos com todo o rigor que a linguagem de programação exige. O Português estruturado está mais próximo do Português narrativo e o pseudo-código está mais próximo da linguagem de programação.

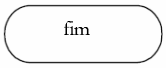
Para além destes, existem vários outros métodos de especificação de algoritmos, incluindo:

- Fluxogramas;
- Diagramas de Nassi-Shneiderman
- Tabelas de decisão;
- Árvores de decisão;
- Pré e pós condições

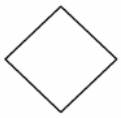
Destes veremos, apenas e para já, os fluxogramas.

Os fluxogramas utilizam um pequeno conjunto de símbolos para representar de uma forma gráfica os algoritmos. Tipicamente, os símbolos usados são:

- Terminador:



- Decisão:



- Entrada/Saída:



- Processamento:



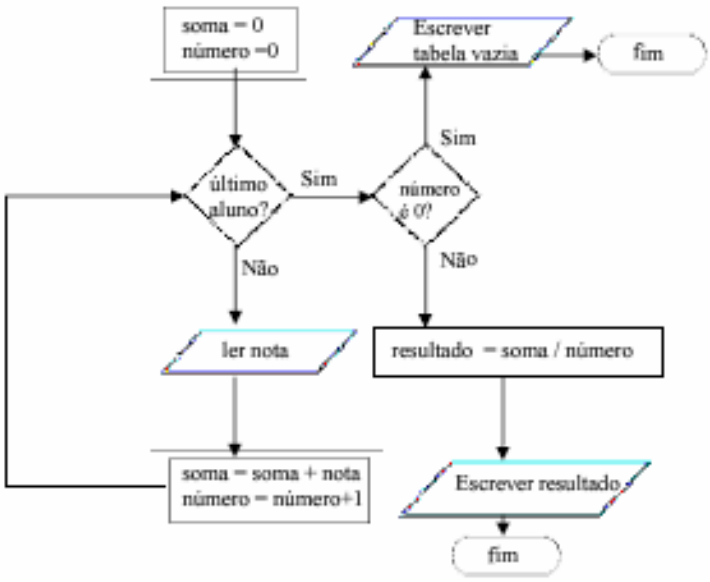
- Conector:



- Ponte de ligação:



O nosso algoritmo **Calcula Média**, apresentado na página 5, pode ser descrito usando um fluxograma da seguinte forma;



Os fluxogramas descrevem de uma forma gráfica o *fluxo de controlo* de um algoritmo ou programa. O fluxo de controlo é a sequência pela qual os passos ou as instruções são executadas.

A descrição oferecida pelos fluxogramas não é estruturada. Os diagramas de Nassi-Shneiderman, foram propostos para obviar este problema e podem ser vistos como fluxogramas estruturados.

Os componentes dos diagramas de Nassi-Shneiderman incluem:

Exercícios

Especifique cada um dos algoritmos desenvolvidos nos exercícios 2 a 8 usando um fluxograma.

Capítulo

II

Codificação de informação

Apresentam-se as unidades de informação e os sistemas de numeração. Introduz-se a noção de tipo de dados.

Os computadores manipulam diversos tipos de informação incluindo: números, letras, imagens, vídeo e som.

Sendo um computador constituído por circuitos electrónicos que funcionam apenas com dois estados; ligado (1) e desligado (0) todo o tipo de informação acabará sempre por ser armazenada como uma sequência de dígitos binários. Assim, toda a informação tem de ser codificada de alguma maneira numa sequência de zeros e uns.

Unidades de informação: bits, bytes, kilobytes, ...

Tal como existem unidades para medir pesos (miligrama, grama, kilograma,...) e distâncias (milímetro, centímetro, decímetro, metro, ...), também existem unidades para medir informação (bit, byte, kilobyte, megabyte, gigabyte, terabyte, ...).

Um bit é a unidade de informação correspondente a um único dígito binário (0 ou 1). Um byte são 8 bits. Um kilobyte são $2^{10} = 1024$ bytes. Um megabyte são 2^{20} bytes (aproximadamente 1 milhão de bytes). Um gigabyte são 2^{30} bytes (aproximadamente 1 bilião de bytes). Um terabyte são 2^{40} bytes (aproximadamente 1 trilião de bytes).

Para simplificar, é costume designar kilobyte por Kbyte ou simplesmente KB. O mesmo se passa para megabyte (MB) e gigabyte (GB)



Para dar uma ordem de grandeza podemos dizer que:

i) uma disquete pode armazenar até 1,44 MB de informação
ii) um CD de música guarda tipicamente 640MB,
iii) um DVD 4,7GB
iv) a capacidade do disco rígido de um computador pessoal actual não será ser inferior a 60 GB.

Números inteiros

A maior parte dos compiladores usam um número fixo de bits para representar números. Estes números tomam valores dentro de limites específicos que dependem do número de bits que usamos para os representar.

Para facilitar a apresentação, vamos considerar um compilador onde os números inteiros são representados usando apenas 3 bits. Os valores possíveis para esses 3 bits são:

000, 001, 010, 011, 100, 101, 110 e 111

O número de inteiros diferentes que poderemos representar usando 3 bits é dado por $2^3=8$. Se em vez de 3 bits tivéssemos 8, então poderíamos representar $2^8=256$ valores diferentes. No caso geral, para um compilador de n bits, *i.e.*, para um compilador que represente inteiros usando n bits, existirão 2^n inteiros diferentes.

Até aqui vimos quanto números diferentes podemos representar com n bits. Teremos que ver agora qual a quantidade representada por um certo número. Isso leva-nos aos sistemas de numeração.

Sistemas de numeração

O sistema de numeração que nós usamos na nossa vida diária é o sistema decimal. Este sistema chama-se decimal, ou de base 10, porque utiliza 10 símbolos diferentes: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. Com estes 10 símbolos somos capazes de construir números tais como 747. O número 747 tem uma sequência de 3 símbolos, dígitos ou algarismos, dois dos quais repetidos (os dois setes). No entanto, o primeiro 7 tem um valor diferente do segundo 7. O primeiro vale 700 (7 centenas) mas o segundo já só vale 7 (7 unidades). De facto,

$$747 = 700 + 40 + 7 = 7*10^2 + 4*10^1 + 7*10^0$$

Portanto, os algarismos representam um valor diferente consoante a sua posição. No sistema decimal, o peso dos algarismos são potências de 10. Todos nós aprendemos isto na escola primária e nada disto é novidade.

Não é por acaso que usamos 10 símbolos ou dígitos diferentes, é porque temos um total de 10 dedos nas mãos. Se em vez de 10 dedos tivéssemos apenas 3, provavelmente o nosso sistema de numeração usaria apenas 3 dígitos diferentes. Como é que seriam os números no sistema ternário ou de base 3?
Antes de responder vamos voltar outra vez ao sistema decimal e ver como é que contamos os números.

```
0 1 2 3 4 5 6 7 8 9 10 11 … 19 20 21 … 99 100 101 …
```

Como só temos 10 símbolos, quando um algarismo chega a 9, temos de fazer "0 e vai um..." Se na escola primária só nos tivessem ensinado 3 símbolos (0,1,2) seria a mesma coisa: quando o algarismo chegasse a 2 teríamos de fazer "0 e vai um..." , pelo que os números seriam:

```
0 1 2 10 11 12 20 21 22 100 101 102 110 111 112 120
121 122 200 201 202 210 …
```

O conceito é exactamente o mesmo que existe no sistema decimal. Por exemplo, o número 121 na base três tem 3 símbolos mas o primeiro 1 tem um

valor diferente do segundo 1. Com um sistema de base 3, o peso associado a cada algarismo é uma potência de 3. Assim:

$$121_{(3)} = 1*3^2 + 2*3^1 + 1*3^0$$

isto é, o número 121 na base 3 corresponde ao número 9+6+1=16 na base 10.

Um dos primeiros sistemas de contagem pode ser visto como um sistema de numeração com apenas um símbolo: as coisas contavam-se com pauzinhos. Os computadores utilizam o sistema de numeração binário ou de base 2. De seguida apresenta-se os números de 0 a 20 na base 10, base 3, base 2 e na base 1.

base 10	base 3	base 2	base 1 (pauzinhos)
0	0	0	
1	1	1	|
2	2	10	||
3	10	11	|||
4	11	100	||||
5	12	101	|||||
6	20	110	||||||
7	21	111	|||||||
8	22	1000	||||||||
9	100	1001	|||||||||
10	101	1010	||||||||||
11	102	1011	|||||||||||
12	110	1100	||||||||||||
13	111	1101	|||||||||||||
14	112	1110	||||||||||||||
15	120	1111	|||||||||||||||
16	121	10000	||||||||||||||||
17	122	10001	|||||||||||||||||
18	200	10010	||||||||||||||||||
19	201	10011	|||||||||||||||||||
20	202	10100	||||||||||||||||||||

Para além dos sistemas binário e decimal, outros sistemas com interesse são os sistema octal (base $8=2^3$) e o hexadecimal (base $16=2^4$), por permitirem conversões imediatas de e para o sistema binário.

As conversões de binário para octal são feitas fazendo grupos de 3 bits da direita para a esquerda, e substituindo cada grupo pelo símbolo octal correspondente. Assim: $101\ 001_{(2)}=51_{(8)}$

Em hexadecimal, o processo é idêntico. A única diferença é que em vez de grupos de 3 bits usam-se grupos de 4. Assim: $101001_{(2)} = 0010\ 1001_{(2)} = 29_{(16)}$

Gamas de variação para inteiros

Podemos agora voltar ao nosso computador de três bits. Já vimos que com três bits podemos representar 8 números diferentes. Mas que números são esses? Como podemos ver na coluna da base 2, com apenas 3 bits para representar inteiros em binário natural, as quantidades representadas começam em 0 e vão até 2^3-1=7. Estes são os limites de variação, ou a gama de valores possíveis, para inteiros sem sinal de 3 bits. Com 8 bits a gama de valores seria de 0 a 2^8-1=255. E com n bits, quais seriam os limites de variação?

O que vimos é válido para inteiros sem sinal. A representação de inteiros com sinal é feita usando aquilo a que se chama representação em complemento para 2.

Voltemos ao nosso compilador de 3 bits. Os valores possíveis para esses três bits continuam a ser os mesmos: 000, 001, 010, 011, 100, 101, 110 e 111. Em complemento para 2, o bit mais à esquerda no número (ou o bit mais significativo) é usado para indicar o sinal do número. Se esse bit for 0 estamos na presença de um número positivo; se esse bit for 1 teremos um número negativo.

Números positivos começam por 0 e os restantes bits representam o valor absoluto do número. Números negativos começam por 1, sendo o seu valor absoluto dado pela diferença entre a quantidade de números que se conseguem representar com os restantes dígitos e o valor em binário natural representado pelos dígitos restantes.

	SINAL	VALOR ABSOLUTO
000	+	0
001	+	1
010	+	2
011	+	3
100	-	4-0=4
101	-	4-1=3
110	-	4-2=2
111	-	4-3=1

Assim, embora um inteiro com sinal, representado usando 3 bits, tenha ainda 2^3=8 valores possíveis, os seus limites de variação são: -(2^2)=-4 e 2^2-1 =3.

Se tivermos 8 bits então a gama de variação vai de –(2^7)=-128 a 2^7-1=127.

No caso geral, um inteiro usando n bits terá 2^n valores diferentes e terá as seguintes gamas de variação:

- Sem sinal: 0 a 2^n-1
- Com sinal: –(2^{n-1}) a 2^{n-1}-1.

Caracteres

Os caracteres 'a', 'b', 'c',... , 'A', 'B', 'C,... ,'+' ,'-' ,'?' ,... costumavam ser representados internamente nos computadores usando o código ASCII (*American Standard Code for Information Interchange*). Neste código, cada caracter é codificado usando 8 dígitos binários. De seguida, apresenta-se o código de alguns caracteres:

caracter	código decimal	código binário
.	.	.
.	.	.
.	.	.
#	35	00100011
$	36	00100100
%	37	00100101
&	38	00100110
'	39	00100111
(	40	00101000
)	41	00101001
*	42	00101010
+	43	00101011
,	44	00101100
-	45	00101101
.	46	00101110

/	47	00101111
0	48	00110000
1	49	00110001
2	50	00110010
3	51	00110011
.	.	.
.	.	.
.	.	.
A	65	01000001
B	66	01000010
C	67	01000011
.	.	.
.	.	.
.	.	.
a	97	01100001
b	98	01100010
c	99	01100011
.	.	.
.	.	.
.	.	.

Como vimos antes, com 8 bits podemos representar 2^8=256 caracteres diferentes.

As linguagens de programação actuais usam o unicode[ii]. O unicode é um código que em vez de 8 usa 16 bits para representar um caracter. Com 16 bits o unicode representa 65536 caracteres diferentes, o que dá para representar praticamente todos os caracteres dos diferentes alfabetos existentes actualmente.

Noção de tipo de dados

Como vimos, a mesma sequência de bits pode representar valores muito diferentes, consoante se trate de inteiros com ou sem sinal, caracteres ou outros *tipos de dados*.

Esta é apenas uma das razões introduzirmos a noção de tipo de dados. Outras razões, talvez até mais importantes, serão apresentadas mais tarde.

Todas as linguagens de programação actuais proporcionam pelo menos um conjunto mínimo de tipos de dados pré-definidos, assim como a capacidade de construir novos tipos, ou tipos definidos pelo utilizador. Mas afinal o que é um tipo de dados?

Definição: Um tipo de dados é um conjunto de valores e uma colecção de operações sobre esses valores.

Exemplo: O tipo de dados `int` consiste do seguinte conjunto de valores

{0, +1, -1, +2, -2, ..., INT_MAX, INT_MIN}

onde INT_MAX e INT_MIN representam os limites superior e inferior da gama de variação dos inteiros. As operações sobre os `int` são muitas e incluem os operadores aritméticos `+`, `-`, `*` e `/`, teste de igualdade e desigualdade, operações de atribuição, etc.

Exercícios

1. Qual é o valor representado por $1011_{(2)}$?
2. Como representar 65 em binário natural? E em octal? E em hexadecimal?
3. Qual é o valor representado por $1011_{(3)}$? E se o número estivesse na base 10?
4. A sequência binária 1011 codifica um inteiro com sinal. Que inteiro representa?
5. Qual é o número mínimo de bits necessário para guardar 9 valores diferentes?

Capítulo

IV

Primeiros programas

Desenvolve-se os primeiros programas. Introduzem-se as noções de tipos primitivos de dados, variáveis, Instruções de atribuição e expressões aritméticas e lógicas. Discutem-se os primeiros aspectos sobre a documentação de código.

O programa seguinte está escrito na linguagem de programação Java.

```
public class PrimeiroPrograma {
        public static void main(String[] args) {
                System.out.print("Primeiro programa em Java.");
        }
}
```

Ao ser executado, o programa escreve o seguinte no ecrã:

```
Primeiro programa em Java.
```

Como é que podemos confirmar isso? Uma das formas mais simples consiste em[iii]:

1. copiar o programa para um editor de texto e gravar o ficheiro correspondente com o nome PrimeiroPrograma.java Por razões que se tornaram claras mais à frente, é nesta fase muito importante reproduzir *exactamente* o que está escrito. Por exemplo, é proibido trocar minúsculas por maiúsculas e vice-versa.

2. Depois teremos de compilar o programa usando o compilador de Java, o javac. Para isso, executa-se na linha de comandos:

```
>javac PrimeiroPrograma.java
```

Como resultado, o compilador gerará o ficheiro PrimeiroPrograma.class que contém código executável pela máquina virtual do Java. O código executável ficará guardado no ficheiro PrimeiroPrograma.class

3. Finalmente executa-se o programa, chamando a máquina virtual com o nome do nosso ficheiro à frente. Isto é, na linha de comandos escreve-se:

```
>java PrimeiroPrograma
```

Explicação do primeiro programa

Na nossa sociedade, o engenheiro projecta, o médico cura e o professor explica. Cada uma destas actividades é da responsabilidade de uma profissão particular. Da mesma forma, numa linguagem de programação orientada por objectos, como o Java, cada actividade deve ser da responsabilidade de uma *classe*.

Por exemplo, as funções matemáticas, como as funções trignométricas e logarítmicas, são todas da responsabilidade da classe Math. Se quisermos usar uma destas funções, por exemplo para calcular a raiz quadrada de 2, escrevemos Math.sqrt(2). As restantes funções têm os nomes habituais, como abs, cos, log, log10 ou sin, para valor absoluto, coseno, logaritmo nepreano, logaritmo de base 10, ou seno de um número, respectivamente. Os números irracionais π e e também podem ser encontrados nesta classe e poderão ser usados escrevendo Math.PI e Math.E, respectivamente.

Em termos gerais, uma classe pode ser vista como um mecanismo que permite ao programador criar um novo tipo de dados. Sendo a noção de classe uma noção central em Programação Orientada por Objectos, iremos ter oportunidade de, mais à frente, aprofundar esta noção.

Posto isto, não é portanto de estranhar que no nosso primeiro exemplo de programa, tivéssemos definido uma classe chamada PrimeiroPrograma que fosse responsável por escrever a nossa mensagem. A palavra public especifica a *visibilidade* da classe. Grosso modo, indica quem poderá usar a classe.

Neste momento, interesse reter que a expressão

```
public static void main(String[] args)
```

identifica o programa principal. É aqui que a execução do programa começa, quando o mandamos executar. Por outras palavras, a máquina virtual Java procura por main na classe com o mesmo nome do ficheiro onde escrevemos o programa. Depois disto, a execução continua a partir da primeira instrução de main.[iv]

As chavetas { e }, assinalam o início e o fim de um *bloco* de instruções. Neste programa servem para indicar onde começa e acaba a classe, assim como onde começa e acaba o programa principal.

Está bem de ver que é a instrução

```
System.out.print("Primeiro programa em Java.");
```

que escreve a *cadeia de caracteres* "Primeiro programa em Java." no ecrã.

Segundo programa

No nosso segundo programa em Java vamos declarar, iniciar e escrever no ecrã uma variável do tipo inteiro.

```
public class SegundoPrograma {
    public static void main(String [] args) {
        int i=2;
        System.out.print("Programa No " + i + " em Java.");
    }
}
```

Verifique o que este programa escreve no ecrã.

Este programa já começa a apresentar uma estrutura típica: o programa principal começar por declarar e iniciar a variável i, seguindo-se depois a instrução de escrita.

Da mesma forma que as orações em Português terminam com um ponto final, todas as instruções terminam com ponto e vírgula (;) Por este motivo este carácter chama-se o carácter terminador.

A declaração de uma variável é feita sempre como indicado acima: primeiro o tipo (int), depois o nome da variável (i), segue-se o terminador (;) ou uma inicialização, como no exemplo. Aos nomes das variáveis também se chamam identificadores.

Repare-se ainda como o + torna possível juntar cadeias de caracteres e valores de variáveis.

Tipos primitivos de dados

Em Java existem 8 tipos primitivos (ou elementares) de dados: byte, short, int, long, float, double, char e boolean.

Para guardarmos inteiros temos o byte, short, int e long, estando a diferença no número de bytes disponíveis para armazenamento: 1, 2, 4 e 8 bytes, respectivamente. Para guardarmos reais, temos o float e o double; sendo também aqui a diferença no número de bytes disponíveis para cada tipo: 4 e 8 bytes, respectivamente. O tipo char guarda caracteres. Podemos referir-nos ao carácter a escrevendo

```
char c = 'a';
```

Finalmente, temos o tipo **boolean** que guarda valores lógicos, *i.e.*, **true** e **false**. Como só queremos distinguir entre dois valores possíveis, então as variáveis do tipo **boolean** usam apenas um bit.

A declaração de variáveis de qualquer um destes tipos é feita sempre como indicado no programa acima.

As palavras **byte**, **short**, **int**, **long**, **float**, **double**, **char**, **boolean**, **true** e **false** são, todas elas palavras reservadas, *i.e.*, o seu significado não pode alterado. Por exemplo, não poderemos dar o nome **byte** a uma variável; isso seria assinalado como um erro.

Programa para somar 2 números inteiros

Para compreendermos completamente o que significa uma variável em programação e vermos mais alguns aspectos elementares do Java vamos escrever um programa que some dois inteiros.

```
public class Soma2int {
  public static void main(String[] args) {
        int a, b, s;
        a=7; b=18;
        s=a+b;
        System.out.print(a + " + " + b + " = " + s );
  }
}
```

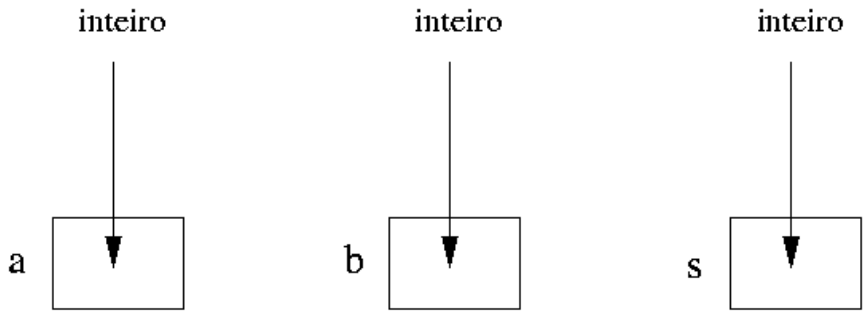
Ao ser executado, o programa vai produzir o seguinte resultado: 7 + 8 = 25. Verifique!

Explicação do programa

O programa principal começa por declarar três variáveis do tipo inteiro:

```
int a, b, s;
```

Podemos pensar numa variável como se fosse uma caixa onde se pode guardar um valor.

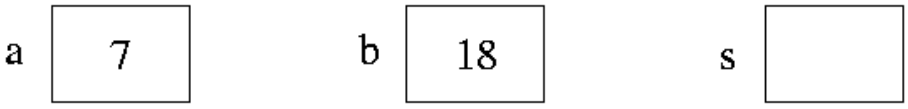


Ao ser declarada, cada variável reserva para si espaço na memória do computador, onde irá guardar os seus valores. Cada variável tem um tipo de dados associado. Como vimos, é o tipo que vai determinar a quantidade de memória reservada para a variável.

Depois da declaração das variáveis, o programa tem 4 instruções que são executadas sequencialmente, uma após outra. Depois de executada a primeira dessas instruções, o estado das variáveis passa a ser:



e depois da segunda será:



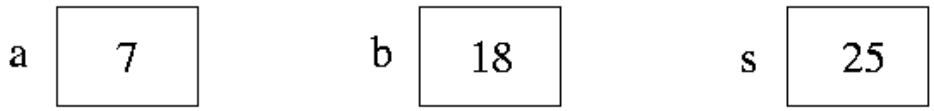
A terceira dessas instruções é uma *instrução de atribuição*:

```
s = a + b;
```

Neste tipo de instrução, aquilo que está no lado esquerdo do sinal de igual é sempre uma variável. Aquilo que está do lado direito do sinal de igual é uma expressão. Ao encontrar este tipo de instrução, o computador efectua o cálculo da expressão do lado direito (`a + b`) e coloca o resultado dentro da variável

que está no lado esquerdo (s). Por outras palavras, atribui-se à variável s o valor de a + b.

Após a instrução de atribuição o estado das variáveis é:



A última instrução do programa é uma operação de escrita, como outras que já vimos anteriormente.

O programa acima soma 2 números inteiros. Se o utilizador introduzir um número real, por exemplo 3.7, o programa não vai ter o comportamento esperado. Que alterações teríamos que fazer ao programa para que fosse capaz de soma 2 números reais?

O programa que acabamos de ver apresenta uma forte limitação. De cada vez que queremos somar dois números diferentes temos que editar o código Java, alterar os números e recompilar o programa. Esta situação não é aceitável. Para a resolver, devemos desenvolver o programa de tal forma que este solicite ao utilizador os números a somar.

```
// Versão 2 da classe Soma2int
import java.util.Scanner;

public class Soma2int {
   public static void main(String[] args) {
            int a, b, s;
            Scanner sc = new Scanner(System.in);
            System.out.print("Primeiro operando: ");
            a=sc.nextInt();
            System.out.print("Segundo operando: ");
            b=sc.nextInt();
            s=a+b;
            System.out.print(a + " + " + b + " = " + s );
   }
}
```

A função sc.nextInt() espera até que o utilizador escreva um inteiro usando o teclado. Após o utilizador carregar na tecla <enter>, a função retorna o valor inteiro lido. Os restantes tipos de dados são lidos recorrendo a funções semelhantes.

Exercícios

1. Defina e inicialize a variável v, de forma a que o seu conteúdo seja representado por 01110101;
2. Escreva um programa que calcule a área de um circulo ($A = \pi r^2$; sendo r o raio).
3. Escreva um programa que, a partir dos números H e M, representando horas e minutos, respectivamente, calcule o número de minutos passados desde as 0 horas.
4. Escreva um programa que converta graus Centigrados, C, em graus Fahrenheit, F ($F = 9/5\ C + 32$).
5. Escreva um programa que calcule o perímetro de um rectângulo de comprimento c e largura l.

Primeiros aspectos sobre a documentação de código

Uma outra novidade que o último programa que desenvolvemos apresenta é o comentário inicial:

```
// Versão 2 da classe Soma2int
```

Um comentário é uma indicação para o programador. Os comentários são literalmente ignorados pelo compilador, *i.e.*, não têm qualquer influência na execução do programa. Para que servem então os comentários? Para ajudar o programador a lembrar-se a si, e quem mais tiver que ler o seu código, de aspectos importantes tais como, o autor do código, o seu contacto, a versão actual, do estado do programa num determinado momento ou ainda das razões que o levaram a tomar uma dada opção. Deste ponto de vista os comentários são uma ferramenta muito importante para melhorar a *legibilidade* do código, *i.e.*, para aumentar a nossa capacidade de compreender o código. Note-se que é

completamente desnecessário adicionar comentários sobre a própria linguagem de programação. Este tipo de informação está disponível a todos e em vários sítios, incluindo nos manuais da linguagem e no próprio ambiente de desenvolvimento.

No comentário acima, o compilador ignora tudo quanto esteja à direita dos caracteres //. Outro exemplo onde este tipo de comentário pode ser muito útil:

```
…
y = x * x +1;
z = Math.sqrt(y);  // Note-se que y é sempre maior do que 0
…
```

Relembra-se que a função Math.sqrt(y) retorna um double com a raiz quadrada de y e que a raiz quadrada só está definida para os números não negativos. Se não tivéssemos a certeza de que y é sempre positivo teríamos que testá-lo antes de chamar a função que calcula a raiz quadrada. Se não o fizéssemos poderíamos ter um *erro de execução*, *i.e.*, um erro que aconteceria enquanto o programa está em funcionamento.

Para além do comentário de linha, que vimos anteriormente, estão também disponíveis os comentários de bloco. Este tipo de comentário começa por /* e acaba com */. Naturalmente que entre o início e o fim do comentário poderemos ter tantas linhas quantas as necessárias, como no seguinte exemplo:

```
/* Versão obsoleta da classe Soma2int
public class Soma2int {
public static void main(String[] args) {
        int a, b, s;
        a=1;
        b=2;
        s=a+b;
        System.out.print(a + " + " + b + " = " + s );
 }
}
*/
```

Não podemos é ter comentários de bloco dentro de comentário de bloco. Para além destes 2 tipos de comentários o Java dispõe ainda dos comentários de documentação de que falaremos mais à frente.

Já que falamos de legibilidade podemos acrescentar desde já mais alguns pontos importantes:

1. Os nomes que dermos às classes e variáveis devem ser tão informativos quanto possível;

   1.1. No caso de uma classe, o seu nome deve dar uma indicação tão clara e concisa quanto possível de qual é a sua responsabilidade.

   1.2. No caso de uma variável, o seu nome deve indicar de forma clara e concisa, qual o significado dos valores nela armazenados.

   É importante notar que os nomes (ou identificadores) são sempre constituídos por uma sequência de caracteres que começam por uma letra e não têm espaços. Quando for útil usar duas ou mais palavras como identificador podemos usar o carácter _ para juntar as palavras. Se preferirmos podemos igualmente juntar as várias palavras capitalizando a inicial de cada uma. Alguns exemplos onde estas indicações podem ser úteis:

```
long total_numeros_telefone;
long totalNumerosTelefone;
```

   No que se segue adoptaremos a seguinte convenção: os nomes das variáveis começam por minúsculas e os nomes das classes começam por maiúsculas.

2. O código deve estar indentado (ou justificado) de tal forma que seja imediato para o leitor identificar o início e o fim de cada um dos blocos do código.

Nos exemplos apresentados até aqui, tivemos a preocupação de seguir estas regras. E se não as tivéssemos seguido, os programas não funcionariam?

Funcionavam, mas a sua compreensão seria certamente mais difícil, senão vejamos:

```
/* Versão 2 da classe Som2int escrita sem qualquer tipo de preocupação com legibilidade*/ public class Soma2int {public static void main(String[]args) {int a, b, s; Scanner sc = new Scanner(System.in); System.out.print("Primeiro operando:"); a=sc.nextInt(); System.out.print("Segundo operando:"); b=sc.nextInt(); s=a+b; System.out.print(a + " + " + b + " = " + s );}}
```

O código acima tem exactamente o mesmo significado para o compilador do que o código da primeira página. No entanto, para nós é muito mais difícil de compreender. Sendo a compreensão mais difícil isso tornaria também mais difícil a sua manutenção, *i.e.*, a sua eventual correcção ou alteração.

Expressões

Uma expressão é uma sequência bem formada de operandos e operadores. O resultado da avaliação de uma expressão é um valor. Já vimos alguns exemplos de expressões aritméticas. A expressão `a + b` é uma expressão aritmética envolvendo o operador aritmético soma +. Os operadores aritméticos disponíveis em Java são:

Operadores aritméticos	*Significado*	*Exemplo*
+, -, *, /, %	Operações aritméticas usuais, sendo % o operador módulo que retorna o resto da divisão inteira dos seus operandos.	`int i = 5 % 2;  // i vale 1`
++a, --a	Incremento, decremento prefixo. O valor da variável `a` é incrementado (ou decrementado) e só depois é que é devolvido.	`int i, a=0;` `i = ++a; // i vale 1, a vale 1`
a++, a--	Incremento, decremento sufixo. O valor da variável `a` é devolvido e só	`int i, a=0;` `i = a--;  // i vale 0, a vale -1`

	depois é incrementado (ou decrementado)	

Certas linguagens como o C, C++ e o Java disponibilizam os chamados operadores de auto-atribuição. Estes operadores são convenientes por que permitem uma escrita concisa de instruções de atribuição. Por exemplo, a instrução

```
a = a * 3;
```

pode ser escrita abreviadamente como:

```
a *= 3;
```

À primeira vista, as vantagens parecem não ser muitas, mas são. Desde logo, a instrução abreviada é mais eficiente do que a outra[v]. Depois a forma abreviada, porque mais simples, torna-se mais legível nos casos não triviais.

Outros operadores de auto-atribuição disponíveis são: `+=`, `-=`, `*=`, `/=`, `%=` cujo significado é evidente.

O Java atribui prioridades (ou precedências) aos vários operadores de forma a que o resultado da avaliação de uma expressão Java seja igual ao resultado da avaliação da expressão matemática correspondente. Assim:

```
x = 1 + 2 * 3;
```

atribui a `x` o valor `7`, tal como seria de esperar, já que em matemática fazem-se primeiro as multiplicações e divisões e só depois as somas e subtracções. Também, tal com em Matemática, podemos usar parêntesis curvos para alterar prioridades e numa expressão com vários operadores aritméticos, todos com a mesma prioridade, avalia-se a expressão da esquerda para a direita.

As expressões lógicas são constituídas por operadores lógicos ou por operadores relacionais. O resultado da avaliação de uma expressão lógica é true ou false, *i.e.*, um valor booleano.

Operadores lógicos	*Significado*	*Exemplo*
&, &&	Conjunção ou e-lógico. Ambos os operadores retornam true se e só se ambos os operandos forem true; retornam false caso contrário. O operador & avalia sempre ambos os operandos. O operador && só avalia o segundo operando se necessário, *i.e.*, se o primeiro operando for true.	boolean a = true, b = false, c; c = a && b; // c vale false
|, ||	Disjunção ou ou-lógico. Ambos os operadores retornam false se só se ambos os operando forem false; retorna true caso contrário. O operador || só avalia o segundo operando se necessário, *i.e.*, se o operando esquerdo for false.	boolean a = true, b = false, c; c = a || b; // c vale true
^	Ou-exclusivo. Retorna true se e só se os seus argumentos tiverem valores lógicos diferentes; Retorna false caso contrário	boolean a = true, b = true, c; c = a ^ b; // c vale false
!	Negação. Operador unário. A negação de false é true e a negação de true é false.	boolean a= true, c; c = !a; // c vale false

Operadores relacionais	*Significado*	*Exemplo*
<	Menor. Retorna true se e só se o	int i = 0;

	operando esquerdo for menor do que operando direito; retorna **false** caso contrário.	float f = 1.0f; boolean b = (i < f); // b vale true

<=	Menor ou igual. Retorna **true** se e só se o operando esquerdo for menor ou igual ao operando direito; retorna **false** caso contrário	int i = 1;
		double d = 0.0;
		boolean b = (i <= d); // b vale false

>	Maior. Retorna **true** se e só se o operando esquerdo for maior do que operando direito; retorna **false** caso contrário.	short s = 10;
		int i = 1;
		boolean b = (s > i); // b vale true

>=	Maior ou igual. Retorna **true** se e só se o operando esquerdo for maior ou igual ao operando direito; retorna **false** caso contrário.	short s = 10;
		int i = 10;
		boolean b = (s <= i); // b vale true

==	Igual. Não confundir com o operador atribuição (=). Retorna **true** se e só se ambos os operandos forem iguais; retorna **false**, caso contrário.	short s = 0x10;
		int i = 16;
		boolean b = (s = i); // b vale true

!=	Diferente. Retorna **true** se e só se ambos os operandos forem diferentes; retorna **false**, caso contrário.	int i = 1;
		float f = 1.0f;
		boolean b = (i != f); // b vale false

Exercícios

1) Quais são os resultados mostrados pelas seguintes instruções?

   a) int x;

      i) x = -3+4*5–6; System.out.println(x);

      ii) x = -3 *4 %–6; System.out.println(x);

      iii) x = (7 + 6) % 5 / 2; System.out.println(x);

b) `int x = 2, y, z;`

i) `x *= 3 + 2; System.out.println(x);`

ii) `x *= y = z = 4; System.out.println(x);`

iii) `x = y == z; System.out.println(x);`

2) Dadas as seguintes atribuições

```
byte umByte = 10;
boolean a=false, b=false;
```

Calcule o valor das seguintes expressões lógicas.

a) `(umByte > 10) && !a`

b) `(!a^b) && (a || !b)`

c) `!(a || b) == (!a && !b)`

3) Escreva uma expressão que permita verificar se o valor da variável **x** é positivo mas menor do 10.

4) Escreva uma expressão que permita verificar se um dado inteiro é par ou impar.

Capítulo

V

Estruturas de controlo de fluxo

Apresentam-se as estruturas de controlo, designadamente apresentam-se a sequência, a instrução de selecção simples if, a instrução de selecção em alternativa if else, a instrução de selecção múltipla, e os iteradores.

Como vimos, quando abordamos a resolução de problemas pode mostrar-se que as sequências, selecções e iterações são suficientes para definir qualquer algoritmo. Sendo um programa uma tradução de um algoritmo para uma linguagem de programação, é conveniente que a linguagem de programação permita a tradução directa dessas estrutura de controlo.

As linguagens de programação imperativas, sejam estruturadas ou orientadas por objectos, dispõem desta facilidade. Assim será possível controlar o fluxo de execução do programa usando directamente sequências, selecções e iterações. Chama-se fluxo de execução de um programa à sequência pela qual as suas instruções são executadas.

Até aqui, vimos apenas exemplos de programas com um fluxo de execução sequencial, *i.e.*, as instruções são executadas umas a seguir às outras, pela ordem que aparecem no programa.

Se tomarmos como exemplo o programa que calcula a área do círculo, pedido num dos exercício anteriores:

```
// Programa para cálculo da área A, de um círculo de raio r: A = pi * r *r
//  versão 1
```

```
import java.util.*;
public class AreaCirculo {
        public static void main (String [] args) {
                float raio;
                double area;
                Scanner sc = new Scanner(System.in);
                System.out.print("Escreva o valor do raio: ");
                raio = sc.nextFloat();
                area = Math.PI * raio * raio;
                System.out.print("Área do circulo de raio "+raio+ "é " + area);
        }
}
```

verificamos que o programa é executado da seguinte forma:

1º Declara a variável raio;

2º Declara a variável area;

3º Escreve no ecrã: Escreva o valor do raio;

4º Atribui à variável raio o valor entrado pelo utilizador;

5º Calcula a área do círculo e atribui-a à variável area;

6º Escreva no ecrã, o valor calculado;

7º Termina a execução.

No que se segue, estudaremos as estruturas de controlo que permitem alterar a execução puramente sequencial. Estas estruturas vão permitir-nos escrever programas verdadeiramente interessantes.

A instrução de selecção simples if

As instruções de selecção são das estruturas mais utilizadas no controlo do fluxo de programa. A mais simples dessas instruções tem a seguinte sintaxe – No anexo A podemos ver o que é a sintaxe de uma linguagem e como a podemos descrever:

if (***expressão_lógica***) ***instrução***

onde ***expressão_lógica*** representa uma expressão lógica e ***instrução*** representa uma instrução que poderá ser simples ou composta. Uma instrução composta não é mais do que um bloco de instruções simples. O bloco começa por { e acaba

com }. Para além de instruções, um bloco pode conter, no seu início, a declaração de variáveis.

O significado desta instrução é o seguinte: Se e só se ***expressão_lógica*** resultar no valor lógico true, então a próxima instrução a ser executada será a instrução ***instrução***. Caso contrário, tudo funciona como se a instrução ***instrução*** não fizesse parte do programa.

Para dar um exemplo, pensemos novamente no caso do programa que calcula a área de um círculo. Será que faz sentido calcular a área mesmo se o utilizador der um valor negativo para o raio? A resposta é não. Matematicamente, o raio é definido como uma distância e uma distância é, por definição, não negativa. Deste ponto de vista, a versão 1 que apresentamos para este programa deverá ser re-escrita. A versão seguinte do programa, representa uma primeira melhoria face à versão anterior.

```
// Programa para cálculo da área A, de um círculo de raio r : A = pi * r *r
//  versão 2 - O raio terá que obedecer à condição r>= 0
import java.util.*;
public class AreaCirculo {
        public static void main (String [] args) {
                float raio;
                double area;
                Scanner sc = new Scanner(System.in);
                System.out.print("Escreva o valor do raio: ");
                raio = sc.nextFloat();
                if(raio>=0) {
                        area = Math.PI * raio * raio;
                        System.out.print("Área do circulo de raio "+raio+ "é " + area);
                }
        }

}
```

Nesta versão, se e só se o raio for positivo ou nulo é que o programa executa a instrução composta

```
        {
                area = Math.PI * raio * raio;
```

```
            System.out.print("Área do circulo de raio "+raio+ "é " + area);
    }
```

Caso contrário esta instrução composta é simplesmente ignorada e o programa termina imediatamente.

A instrução de selecção em alternativa if else

Frequentemente fazemos uma coisa se uma dada condição se verificar mas faremos outra, caso a condição não se verifique. Para estes casos, podemos usar a instrução de selecção em alternativa **if else**, cuja sintaxe é:

if (***expressão_lógica***) ***instrução1*** else ***instrução2***

Se ***expressão_lógica*** retornar o valor lógico true então o programa executa a ***instrução1***, senão o programa executa ***instrução2***.

A instrução **if else** funciona do seguinte modo. O valor da expressão é calculado. Se o valor for true, a ***instrução1*** será executada mas a ***instrução2*** não. Se a condição avaliar false, a ***instrução2*** será executada mas a ***instrução1*** não.

Podemos usar esta instrução para melhorar mais um pouco o nosso programa do cálculo da área do círculo. Se o utilizador introduzir um valor negativo para o raio será informado disso antes do programa terminar.

```
// Programa para cálculo da área A, de um círculo de raio r : r>= 0 e A = pi * r *r
//  versão 2.1 – O utilizador é informado se introduzir um raio menor do que 0.
import java.util.*;
public class AreaCirculo {
        public static void main (String [] args) {
                float raio;
                double area;
                Scanner sc = new Scanner(System.in);
                System.out.print("Escreva o valor do raio: ");
                raio = sc.nextFloat();
```

```
            if(raio>=0) {
                area = Math.PI * raio * raio;
                System.out.print("Área do circulo de raio "+raio+ "é " + area);
            }
            else    System.out.println("O raio não pode ser negativo");
        }
    }
```

Portanto, nesta versão se o utilizador inserir um valor positivo ou nulo para o raio o programa calcula a área e apresenta-a no ecrã. Caso contrário, o programa escreve simplesmente a mensagem: O raio não pode ser negativo.

Outro exemplo. Matematicamente define-se sinal de *x*, *S(x)*, para *x* real, da seguinte forma:

$$S(x) = \begin{cases} 1 & \text{se } x > 0 \\ 0 & \text{se } x = 0 \\ -1 & \text{se } x < 0 \end{cases}$$

O programa seguinte apresenta uma implementação possível para esta função.

```
// Programa que calcula o sinal de um número real
import java.util.*;
public class Sinal {
    public static void main (String [] args) {
        float x;
        Scanner sc = new Scanner(System.in);
        System.out.print("Escreva um real: ");
        x = sc.nextFloat();
        if(x>0)
            System.out.print("1");
        else if (x<0)
            System.out.print("-1");
          else
            System.out.println("0");

    }

}
```

Vale a penar reparar que esta implementação é mais eficiente do que aquela que é habitualmente seguida por alguns jovens programadores, designadamente

```
if (x>0) System.out.print("1");
if (x<0) System.out.print("-1");
if (x==0) System.out.println("0");
```

Na implementação acima o programa é forçado a fazer sempre 3 testes. Na implementação da classe Sinal, o programa fará um teste ou dois, no máximo.

Quando misturamos instruções if com instruções if else podem surgir ambiguidades. No código

```
if(a) if (b)
        i++;
else
        --i;
```

Se não fosse dito mais nada não saberíamos a que if pertenceria o else. Esta situação está tipificada com o nome "*dangling else*". A regra para se eliminar esta ambiguidade é a seguinte: O else pertence sempre ao if mais próximo que ainda não tenha else.

Exercícios

1. Desenvolva um programa que calcule o máximo de 2 números reais. Os 2 números devem ser introduzidos pelo utilizador. Exemplo de utilização:

   ```
   Introduza um número: 3 <Enter>
   Introduza outro número: 8 <Enter>
   O máximo é 8.
   ```

2. Desenvolva um programa que calcula o valor absoluto de um número introduzido pelo utilizador. Exemplo de utilização:

   ```
   Introduza um número: -3 <Enter>
   ```

O valor absoluto de -3 é 3.

3. Implemente o algoritmo que desenvolveu para ordenar por ordem crescente três números pedidos ao utilizador (exercício 7 do capitulo III).

4. Desenvolva um programa que apresente:
   a. as raízes reais de um polinómio do segundo grau, caso existam.
   b. As raízes reais ou imaginárias de um polinómio do segundo grau. Exemplo de utilização:

   Introduza os coeficientes de um polinómio de grau 2

   a: 1 <Enter>
   b: 2 <Enter>
   c: 2 <Enter>
   As raízes são -1+i e -1-i

5. Desenvolva um programa que receba 4 valores inteiros e que indique quantos desses valores são pares, quantos são ímpares, quantos são positivos e quantos são negativos.

6. Desenvolva um programa que calcule o máximo de 3 números reais dados pelo utilizador. Exemplo de utilização:

   Introduza um número: 3.2 <Enter>
   Introduza outro número: 8.0 <Enter>
   Introduza mais outro número: 2.8 <Enter>
   O máximo é 8.0

O operador condicional

Uma das formas de encontrar o máximo de três ou mais números, consiste em encontrar máximos sucessivos entre pares de números. Se dois números

estiverem guardados nas variáveis x e y poderemos calcular o máximo escrevendo:

```
if(x>y)
        maximo = x;
else
        maximo = y;
```

Usando o operador condicional poderemos escrever mais abreviadamente

```
maximo = x>y ? x : y;
```

e teremos exactamente o mesmo resultado. O operador condicional, por vezes também chamado o if-aritmético, tem a seguinte forma:

expressão_lógica ? ***expressão1*** : ***expressão2*** ;

Se ***expressão_lógica*** valer true, o operador retorna o valor da ***expressão1*** caso contrário retorna o valor da ***expressão2***.

A instrução de selecção múltipla switch

Temos agora pela frente a tarefa de construir um programa que, dado um índice inteiro entre 1 e 7 apresente o dia da semana respectivo, sendo que o índice 1 corresponde à segunda-feira e o índice 7 corresponde ao domingo.

Este programa, assim como muitos outros, admite várias implementações. Usando a instrução de selecção alternativa if else, que vimos antes, o programa seria:

```
// Programa que dado um índice inteiro entre 1 e 7 apresenta o dia da semana
// correspondente
// Versão 1 – Implementação baseada na instrução de selecção alternativa if else
Scanner sc = new Scanner(System.in);
public class DiaDaSemana {
        public static void main(String [] args) {
                        int i;
                        Scanner sc = new Scanner(System.in);
                        System.out.print("Escreva um numero entre 1 e 7: ");
                        i = sc.nextInt();
                        if(i==1) System.out.println("Segunda-feira");
                           else if (i==2) System.out.println("Terça-feira");
                                else if(i==3) System.out.println("Quarta-feira");
                                   else if(i==4)     System.out.println("Quinta-feira");
                                        else if (i==5) System.out.println("Sexta-feira");
                                          else if(i==6) System.out.println("Sábado");
                                             else if(i==7) System.out.println("Domingo");
                                                else System.out.println("Número inválido");
        }
}
```

Para este tipo de casos, em que dependendo do valor de uma variável podemos fazer várias coisas, temos disponível a instrução de selecção múltipla switch.[vi] Usando esta instrução o nosso programa passaria a ter o seguinte aspecto:

```
// Programa que dado um índice inteiro entre 1 e 7 apresenta o dia da semana
// correspondente
// Versão 2 – Implementação baseada na instrução de selecção múltipla switch
import java.util.*;
public class DiaDaSemana {
        public static void main(String [] args) {
                int i;
                Scanner sc = new Scanner(System.in);
                System.out.print("Escreva um número entre 1 e 7: ");
                i = sc.nextInt();
                switch(i) {
                    case 1:System.out.println("Segunda-feira"); break;
                    case 2: System.out.println("Terça-feira"); break;
                    case 3: System.out.println("Quarta-feira"); break;
                    case 4: System.out.println("Quinta-feira"); break;
                    case 5: System.out.println("Sexta-feira"); break;
                    case 6: System.out.println("Sábado"); break;
                    case 7: System.out.println("Domingo"); break;
                    default: System.out.println("Número inválido");
                 }
           }
}
```

A instrução switch funciona do seguinte modo. Primeiro, o programa calcula o valor da expressão dentro de parentesis logo a seguir à palavra switch. No exemplo acima, é apenas lido o valor da variável i. Baseado nesse valor, o fluxo de programa salta para o caso correspondente. Por exemplo, se i guardar o valor 3, o fluxo salta para a instrução que vem a seguir a case 3:. Depois prossegue com as instruções restantes até ao fim (do corpo) do switch ou até aparecer a instrução break;. A instrução break faz com que o fluxo salte novamente, desta vez para fora do switch, isto é, o programa prossegue com a instrução que aparece imediatamente a seguir à chaveta correspondente ao final da instrução switch. O default: é opcional e só é executado se nenhum dos outros casos ocorrer.

Tudo o que se pode fazer com a instrução switch também se pode fazer com uma sequência encadeada de if-else tal como foi ilustrado no início. No entanto, quando temos muitas opções diferentes, é aconselhável usarmos a instrução switch porque, entre outras razões, torna o programa mais legível.

A descrição formal da instrução switch é dada por:

```
switch ( expressão ) {
        { case literal :  instrução [ break;] }
        [default : instrução [ break;] ]
```

onde ***expressão*** representa uma expressão que avalia um inteiro ou um caracter[vii], ***literal*** é um valor constante e ***instrução*** representa uma instrução que poderá ser composta. As palavras switch, case, break e default são palavras reservadas do Java.

Exercícios

1. Re-escreva o programa que calcula o máximo de três números, usando agora o operador condicional.

2. Descreva por palavras suas o funcionamento do seguinte troço de código:

```
switch(n) {
        case 1: System.out.println("Um"); break;
        case 2: System.out.println("Dois"); break;
        case 3:
        case 4: System.out.print("Três ou quatro: ");
                System.out.println("Nem mais nem menos."); break;
        default: System.out.println("Ou negativo, ou zero, ou muitos ");
}
```

3. Escreva um programa que dê um parecer qualitativo a uma classificação numérica de 0..20 nos seguinte termos: (0..4 -> Mau, 5..9 -> Medíocre, 10..13 -> Suficiente, 14..17 -> Bom , 18..20 -> Muito Bom). Exemplo de utilização:

```
Classificação (0..20):   18 <Enter>
Muito Bom
```

4. Escreva um programa que pede ao utilizador uma operação aritmética e, seguidamente, pede os dois operandos sobre os quais quer realizar essa operação. O programa deverá depois apresentar o resultado da operação no ecrã. O programa não deverá permitir a divisão por zero. Exemplo de utilização:

```
Escolha uma operação ( + - * / ): / <Enter>
Primeiro operando: 7 <Enter>
Segundo operando: 2 <Enter>
7 / 2 = 3.5
```

Iteradores

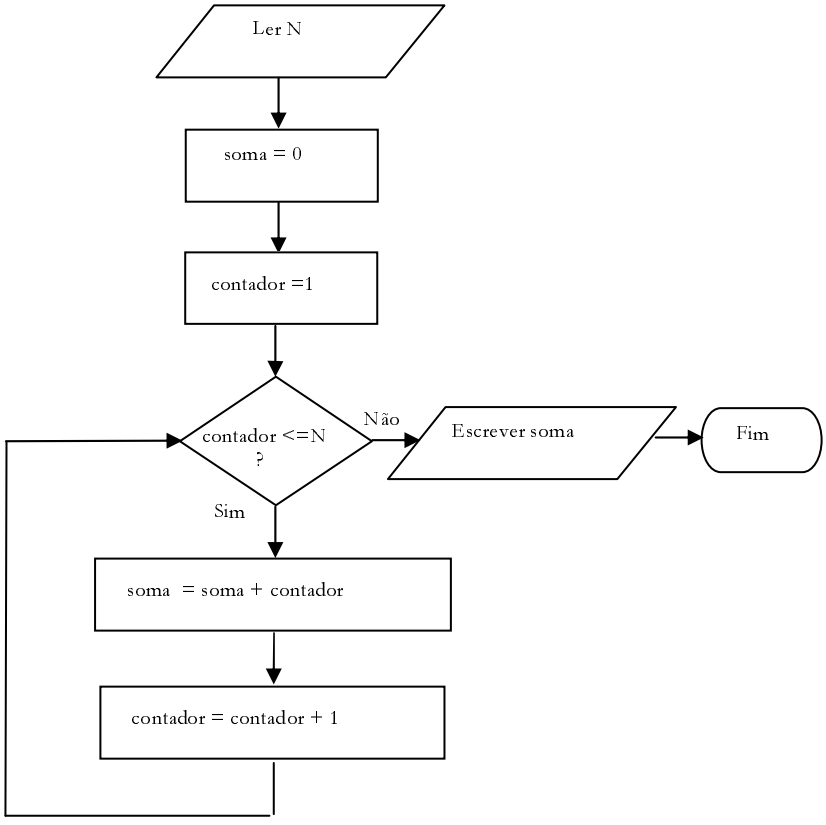
A nossa experiência de programação pode ainda não ser muita, mas já vimos muitos exemplos onde foi necessário repetir, repetir e repetir um conjunto de instruções. Por exemplo, quando abordamos o tema da resolução de problemas, um dos primeiros exercícios que fizemos foi o de calcular a soma dos 10 primeiros números naturais (inteiros positivos).[viii]

Um algoritmo que permite somar os *N* primeiros números naturais é:

Algoritmo para somar os primeiros N naturais

1. Pedir ao utilizador o valor de N
2. Iniciar soma =0
3. Iniciar contador = 1
4. Repetir os passos seguintes enquanto contador <= N
   4.1. Actualizar soma: soma = soma + contador
   4.2. Actualizar contador: contador = contador + 1
5. Escrever soma

O fluxograma correspondente é:



O tipo de repetição representada usada neste algoritmo vai ser necessária muitas e muitas vezes.

Existe uma instrução que permite implementar directamente este tipo de repetição ou de ciclo. Essa instrução é uma das instruções do tipo iterador e chama-se while. Se formos a um dicionário verificamos que iterar significa repetir ou voltar a fazer.

A instrução while

A instrução (às vezes tanta chamada ciclo) while tem a seguinte sintaxe.

while (***expressão_lógica***) ***instrução***

A palavra inglesa while significa *enquanto*. No ciclo while, ***instrução*** representa uma instrução simples ou composta e será executada repetidamente enquanto o valor da expressão lógica ***expressão_lógica*** for true. O programa começa por testar o valor de ***expressão*** e se e só se esse valor for true é que executa ***instrução***. Portanto, se ***expressão_lógica*** for inicialmente false, o ciclo não será executado.

Sabendo isto, a tradução para a linguagem de programação do algoritmo acima é trivial:

```
// Implementação directa do algoritmo que soma os primeiros N naturais
// Versão 1
import java.util.*;
public class SomaN {
        public static void main(String [] args) {
                int n;
                Scanner sc = new Scanner(System.in);
                System.out.print("N: ");
                n = sc.nextInt();
                int soma=0, contador = 1;
                while (contador <= n) {
                        soma = soma + contador;
                        contador = contador + 1;
                }
                System.out.print("Soma: " + soma);
```

```
        }
}
```

Agora até poderíamos exercitar alguns dos operadores da linguagem para simplificar o nosso código:

```
// Implementação do algoritmo que soma os primeiros 10 inteiros positivos
// Versão 2
import java.util.*;
public class SomaN {
        public static void main(String [] args) {
                int n, soma=0, contador = 1;
                Scanner sc = new Scanner(System.in);
                System.out.print("N: ");
                n = sc.nextInt();
                while (contador <= n) soma += contador++;
                System.out.print("Soma: " + soma);
        }
}
```

No exemplo, a variável contador serve para controlar a condição do ciclo while. Este tipo de situação é muito frequente em programação. Ao ser executado o programa faz o seguinte:

```
soma começa por valer 0
contador começa por valer 1
N: 3 <Enter>
n passa a ter o valor 10
   (1 <= 3) ?    // true
   soma = 0 + 1
        = 1
   contador passa a valer 2

   (2 <= 3) ?    // true
   soma = 1 + 2
        = 3
   contador passa a valer 3

   (3 <= 3) ?    // true
   soma = 3 + 3
        = 6
   contador passa a valer 4

   (4 <= 3) ?    // false
   fim do ciclo while
```

Escreve soma: 6

Outro exemplo: pretende-se fazer um programa para escrever no ecrã os primeiros N números naturais pares e os respectivos quadrados. Exemplo de utilização:

```
N: 3 <Enter>
2 4
4 16
6 36
```

Tal como antes este programa utiliza o ciclo while.

```
// Programa que apresenta os primeiros N naturais pares e seus quadrados
public class ParN {
public static void main(String [] args) {
        int n, n2, par = 2;
        System.out.print("N: "); n = Le.umInt();
        n2 = 2*n;
        while (par <= n2) {
                System.out.println(par + " " + par*par);
                par += 2;
        }
}
 }
```

Algumas notas sobre este código:

A variável que controla a condição do ciclo é a variável par. Esta variável começa por tomar o valor do primeiro número par natural (*i.e.*, 2) e em cada iteração toma o valor do número par seguinte, por força da instrução par += 2 (o que é a mesma coisa do que par = par + 2).

O programa utiliza uma variável auxiliar n2 = n*2. Isto evita que o programa tenha que fazer esta multiplicação de cada vez que testa o valor da expressão que controla o ciclo.

EXERCÍCIOS

1. Implemente um programa que escreva no ecrã o caracter '.' o número de vezes pedido pelo utilizador.

2. Desenvolva um programa que escreva a tabuada de um determinado número. Exemplo de utilização:

```
Número inteiro: 9 <Enter>
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
```

3. Escreva um programa que calcule o factorial de um dado número.

4. Escreva um programa que apresente o número de algarismos (ou dígitos) de um número natural n.[ix] Exemplo de utilização:

```
Número inteiro: 123 <Enter>
123 tem 3 dígitos
```

5. Escreva um programa que leia um valor inteiro constituído apenas por zeros e uns e que determine quantos zeros e quantos uns esse valor inteiro possui. Exemplo de utilização:

```
Número com dígitos binários: 101100  <Enter>
3 zeros e 3 uns
```

6. Escreva um programa que leia um valor inteiro e que determine a frequência (i.e., quantidade) de cada um dos seus diferentes algarismos. Exemplo de utilização:

```
Número inteiro: 30032005  <Enter>
zeros: 4
dois: 1
três: 2
cincos: 1
```

7. Implemente um programa que desenhe no ecrã um quadrado usando caracteres '*'. O lado do quadrado deve ser pedido ao utilizador. Sugestão: relembre-se do exercício 1. Exemplo de utilização:

```
lado: 5 <Enter>
*****
*   *
*   *
*   *
*****
```

A instrução for

Além do ciclo while, existem mais dois tipos de ciclos em Java: o ciclo for e o ciclo do-while. As secções seguintes apresentam cada um deles.

O ciclo for é um dos ciclos mais utilizados já que corresponde a uma certa abreviação da instrução while.

A implementação do algoritmo que soma os primeiros N naturais, que vimos anteriormente, pode ser feita usando uma instrução for, da seguinte forma

```
// Implementação directa do algoritmo que soma os primeiros N naturais
// usando a instrução for.
// Versão 1.1
import java.util.*;
public class SomaN {
        public static void main(String [] args) {
                int n, soma=0;
                Scanner sc = new Scanner(System.in);
                System.out.print("N: ");
                n = sc.nextInt();
                for (int contador = 1; contador <= n; contador = contador +1)
                        soma = soma+contador;

                System.out.print("Soma: " + soma);
        }
}
```

A descrição formal da instrução for é:

for (***[inicialização]*** ; ***[teste]*** ; ***[actualização]***) ***instrução***

e é equivalente a:

```
[inicialização]
while ( teste | true ) {
        instrução
        [actualização]
}
```

Caso exista, a instrução ***inicialização*** é executada uma única vez antes do ciclo propriamente dito começar. Em cada iteração do ciclo, o valor da `expressão` ***teste*** é calculado. Caso esta expressão não exista no nosso código, o Java assumirá o valor true. Se este valor for true, é executada instrução seguido da expressão ***actualização***. O ciclo termina quando o teste avaliar false.

Tipicamente, ***inicialização*** é uma instrução de inicialização do ciclo (no nosso exemplo int contador =1), ***teste*** é a condição de terminação do ciclo (no nosso exemplo, contador <= n) e ***actualização*** corresponde a uma instrução que deve ser feita em cada iteração do ciclo (no nosso exemplo, contador = contador +1).

É importante notar que todos os elementos da instrução for são opcionais ou facultativos. Isto torna a instrução muito versátil, *i.e.*, pode ser utilizada em várias situações. Exemplo de ciclo infinito:

```
for(;;)     /* Ciclo infinito */
```

EXERCÍCIO

8. re-escreva, usando uma instrução for, a versão 2.0 do código que calcula a soma dos N primeiros naturais.

O programa que vimos anteriormente que escreve no ecrã os primeiros N números naturais pares e os respectivos quadrados, escrito usando uma instrução for, tem o seguinte aspecto:

```
// Programa que apresenta os primeiros N naturais pares e seus quadrados
// versão 2: usando uma instrução for.
import java.util.*;
public class ParN {
public static void main(String [] args) {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("N: ");
        int n = sc.nextInt();
        for (int n2= 2*n, par=2; par <= n2 ; par += 2)
                System.out.println(par + " " + par*par);
        }
}
```

Repare como na instrução for acima, estamos a inicializar duas variáveis n2=2*n e par=2. Como vimos, esta inicialização será feita uma única vez.

A instrução do-while

Os ciclos while e for a primeira coisa que fazem é testar a condição de terminação. O terceiro e último ciclo testa a condição de terminação apenas após ter feito pelo menos uma iteração e chama-se do-while.

do ***instrução*** while (***expressão_lógica***) ;

Neste ciclo, primeiro a ***instrução*** é executada e só depois é que o valor da expressão é testado. Se este valor for true, o ciclo repete voltando a fazer ***instrução*** e a testar ***expressão_lógica***. Se o valor de ***expressão_lógica*** for falso, o ciclo termina e o programa continua na instrução que aparece imediatamente a seguir ao ciclo. Portanto, a instrução que está dentro do ciclo é executada pelo menos uma vez.

Esta instrução é particularmente útil quando a condição de terminação depende de um valor pedido ao utilizador. Neste caso, este valor terá que ser lido, pelo menos uma vez.

Para dar um exemplo, voltemos ao nosso programa que calcula a área de um círculo. A versão mais recente que desenvolvemos para este programa foi:

```
// Programa para cálculo da área A, de um círculo de raio r : r>= 0 e A = pi * r *r
//  versão 2.1 – O utilizador é informado se introduzir um raio menor do que 0.
import java.util.*;
public class AreaCirculo {
        public static void main (String [] args) {
                Scanner sc = new Scanner(System.in);
                System.out.print("Escreva o valor do raio: ");
                float raio = sc.nextFloat();
                if(raio>=0) {
                        double area = Math.PI * raio * raio;
                        System.out.print("Área do circulo de raio "+raio+ "é " + area);
                }
                else    System.out.println("O raio não pode ser negativo");
        }
}
```

Nesta versão, o programa termina sempre que o utilizador inserir um raio negativo. Se quisermos dar a hipótese ao utilizador de corrigir o seu engano, poderemos usar a instrução do-while, da seguinte forma:

```
// Programa para cálculo da área A, de um círculo de raio r : r>= 0 e A = pi * r *r
// versão 3 – Pede um raio tantas vezes quantas as necessárias até que o
// raio seja >=0.
 import java.util.*;
 public class AreaCirculo {
        public static void main (String [] args) {
                float raio;
                Scanner sc = new Scanner(System.in);
                do {
                        System.out.print("Escreva um valor de raio positivo: ");
                        raio = sc.nextFloat();
                } while(raio < 0);
                //              Aqui garantidamente raio >= 0;
                //              caso contrário o ciclo não teria terminado.
                double area = Math.PI * raio * raio;
                System.out.print("Área do circulo de raio "+raio+ "é " + area);
        }
 }
```

Podemos até refinar este programa de forma a avisar o utilizador sempre que inserir um raio negativo.

```
 // Programa para cálculo da área A, de um círculo de raio r : r>= 0 e A = pi * r *r
 //  versão 3.1 – Pede um raio tantas vezes quantas as necessárias até que o
 // raio seja >=0; O utilizador é informado se introduzir um raio negativo.
 import java.util.*;
 public class AreaCirculo {
        public static void main (String [] args) {
                float raio;
                Scanner sc = new Scanner(System.in);
                do {
                        System.out.print("Escreva um valor de raio positivo: ");
                        raio = sc.nextFloat();
                        if (raio<0) System.out.println("O raio não pode ser negativo");
                } while(raio < 0);
                //      Aqui garantidamente raio >= 0;
                //      caso contrário o ciclo não teria terminado.
                double area = Math.PI * raio * raio;
                System.out.print("Área do circulo de raio "+raio+ "é " + area);
        }
 }
```

Como seria o nosso código se só pudéssemos usar a instrução while ?

O exemplo que acabamos de ver, corresponde à utilização típica da instrução do-while. No entanto, nada nos impede de usar esta instrução em programas como o que calcula a soma dos N primeiros números naturais, embora, para estes casos, a instrução for seja a mais adequada.

```
// Implementação directa do algoritmo que soma os primeiros N naturais
// usando a instrução do-while
// Versão 1.2
import java.util.*;
public class SomaN {
        public static void main(String [] args) {
                Scanner sc = new Scanner(System.in);
                System.out.print("N: ");
                int n = sc.nextInt();
                int contador =1, soma=0;
                do{
                        contador = contador +1;
                        soma = soma+contador;
                }
                while(contador <= n);
                System.out.print("Soma: " + soma);
        }
}
```

Instruções de salto incondicional

As instruções break e continue são chamadas instruções de salto incondicional, pelas razões que veremos já de seguida:

break;

Só é permitida na instrução switch, while, for e do-while e serve para passar o controlo para a primeira instrução a seguir ao do corpo da instrução que a contenha. Vimos a utilização típica desta instrução quando abordamos o switch.

continue;

Só é permitida no corpo dos iteradores (while, for ou do-while). Interrompe a iteração actual, passando para a iteração seguinte. Raramente usaremos esta instrução por que diminui a legibilidade do código.

EXERCÍCIOS

9. Repita os exercícios 1 a 5 deste capítulo, usando instruções **for**.

10. Desenvolva um programa que pede um número natural no sistema de numeração decimal e converte-o para binário. O programa deverá validar a sua entrada. Exemplo de execução:

```
Número natural:  9 <Enter>
 9 em binário: 1001
```

11. Altere o programa anterior de forma a converter o número pedido para qualquer outra base. Exemplo de execução:

```
Número: 9 <Enter>
Base de conversão: 8 <Enter>
 9 na base 8: 11
```

12. Escreva um programa que vai pedindo números ao utilizador até que este introduza o número -1. No fim, o programa deve apresentar a média dos números introduzidos (excluindo o -1). Exemplo de utilização:

```
Insira uma série de números e termine-a com -1.
: 5 <Enter>
: 11 <Enter>
: 6 <Enter>
: 8 <Enter>
: -1 <Enter>
Média: 7.5
```

13. Modifique o programa anterior, de modo a apresentar o mínimo, máximo e média dos valores inseridos. Exemplo de utilização:

```
Insira uma série de números e termine-a com -1.
: 5 <Enter>
: 11 <Enter>
: 6 <Enter>
: 8 <Enter>
: -1 <Enter>
Mínimo: 5
Média: 7.5
```

Máximo: 11

14. Escreva um programa que verifique se um número natural é ou não primo (um número natural é primo se só for divisível por 1 ou por si próprio). O programa deverá validar a sua entrada. Exemplo de utilização:

    Número natural: 4 <Enter>
    4 não é primo

15. Escreva um programa que calcule todos os divisores de um número natural. O programa deverá validar a sua entrada. Exemplo de utilização:

    Número natural: 24 <Enter>
    Divisores de 24: 1 2 3 4 6 8 12 24

Apêndice A – O formalismo EBNF

Uma linguagem de programação é um sistema formal descrito pela sua sintaxe e pela sua semântica. A sintaxe define o alfabeto e as frases pertencentes à linguagem. A semântica define o significado de cada uma dessas frases.

A sintaxe pode ser descrita por uma gramática (independente de contexto). Uma gramática inclui um conjunto de símbolos terminais (*i.e.*, que pertencem à linguagem), um conjunto de símbolos não terminais e um conjunto de regras sintácticas (produções ou derivações). O formalismo EBNF – *Extended Backus Naur Formalism* é um dos formalismos mais utilizados para descrever gramáticas.

EBNF – *Extended Backus Naur Formalism*

Uma notação EBNF é uma sequência regras. Cada regra tem uma parte esquerda, que define o seu nome, um sinal ::=, e pela parte direita, que define a sintaxe de uma linguagem em termos de símbolos terminais ou de símbolos não terminais da linguagem (*i.e.*, outras regras). Na especificação das regras usaremos as seguintes convenções:

- Os símbolos terminais são escritos em minúsculas;
- Os símbolos não terminais são escritos em ***itálico e negrito***;
- A barra vertical **|** separa alternativas;
- Os parênteses curvos ***(*** e ***)*** agrupam;
- Os parênteses rectos ***[*** e ***]*** indicam opcionalidade;
- As chavetas ***{*** e ***}*** identificam símbolos que podem ocorrer 0 ou mais vezes; Em vez das chavetas, por vezes, usa-se o *******.
- As reticências ***...*** são usadas para indicar de-até
- As aspas ***" "*** denotam comentários
- Os sinais ***<*** e ***>*** identificam teclas

Exemplo de especificação de refeições usando EBNF:

entrada ::= camarão **|** ostras.

sobremesa ::= gelado **/** ***fruta***.
fruta ::= maça **/** laranja **/** cerejas
prato ::= bife **/** filetes
vegetais ::= brocolos **/** cenouras
refeição ::= ***[entrada] prato (***batatas | arroz***) {vegetais} [sobremesa]***

Refeições possíveis:

*i.*bife batatas
*ii.*camarão filetes arroz brocolos brocolos brocolos cenouras gelado
*iii.*ostras bife arroz cerejas

Um identificador é o nome dado a um elemento de programa, como as variáveis ou as classes. Em EBNF, a especificação de identificador para a linguagem Java é dada por:

identificador ::= ***caracter { caracter | digito }***
caracter ::= ***letra*** **/** _ **/** $
digito ::= 0 **/** 1 **/** … **/** 9
letra ::= a **/** b **/** ....**/** z **/** A **/** B **/** **...** **/** Z

Desta descrição é agora claro que um identificador começa sempre por uma letra, caracter sublinhado _ ou pelo cifrão $. É também claro que um identificador não contem espaços, nem nenhum outro tipo de caracteres para além dos enumerados acima..

Capítulo

VI

Modularidade de funções

Introduz-se a noção de função em programação assim como alguns conceitos fundamentais com ela relacionados. Faz-se uma primeira referência à Qualidade de funções e à recursividade.

Quando escrevemos

```
Math.sqrt (x*x+1)
```

estamos a *utilizar*, a *chamar* ou a *invocar* a função raiz quadrada que está definida na classe Math.

Quando escrevemos

```
public static void main(String[] args) {
        System.out.print("Primeiro programa em Java.");
  }
```

estamos a *definir* a função principal de uma dada classe. Como vimos, a palavra-chave main significa principal em Inglês.

Portanto, já tivemos oportunidade de utilizar e de definir funções. O que poderemos então dizer mais sobre funções que nos seja útil? Várias e importantes coisas. Questões como o que é uma função? Que características deve ter uma *boa* função? ou Por quê usar funções? Questões como estas têm que ser respondidas.

Para nos ajudar na nossa discussão vamos escrever um programa que calcule o número de combinações de n elementos agrupados m a m, C_m^n que é dado pela seguinte formula:

$$C_m^n = \frac{n!}{m!(n-m)!}$$

Como os elementos de programação de que já dispomos podemos escrever o seguinte programa:

```
// Programa que calcula o número de combinações de n elementos agrupados m a m
// Versão 0
import java.util.*;
public class Comb {
        public static void main(String [] arg) {
                int n, m;
                Scanner sc = new Scanner(System.in);
                System.out.print("N: ");
                n = sc.nextInt();
                System.out.print("M: ");
                m = sc.nextInt();
                int i, fn=1;
                for(i=1; i<=n; i++)                     fn = fn*i;
                int fm=1;
                for (i=1; i<=m; i++)        fm = fm*i;
                int nm= m-n, fnm=1;
                for(i=1; i<= nm; i++)       fnm = fnm*i;
                int comb = fn / (fn*fnm);
                System.out.print(comb);
        }
}
```

Se tivéssemos usado funções:

1. O código que calcula o factorial teria sido escrito uma única vez e poderia ser usado tantas vezes quantas as necessárias. Afinal de contas não é isso que fazemos com as funções de entrada e saída? Teríamos portanto usado uma das formas de reutilização de código. Portanto as *boas* funções (*i.e.*, as funções de *Qualidade*) permitem reutilizar código.
2. O programa ficaria mais simples e mais fácil de compreender, *i.e.*, ficaria mais *legível*. Em vez de três ciclos for, que fazem todos a mesma coisa, teríamos três chamadas à mesma função. Portanto as boas funções aumentam a legibilidade do código.

3. A *manutenção*, i.e., a correcção ou actualização do programa seria também mais fácil. Se agora repararmos que o ciclo for pode começar em 2, em vez de começar em 1, como está escrito acima e quisermos corrigir isso, teremos de corrigir cada um dos três ciclos. Se tivéssemos usado funções, esta mesma correcção seria feita num único e bem definido sítio: no corpo da função que implementa o factorial. Portanto as boas funções facilitam a manutenção do código.
4. Poderíamos ter repartido melhor o trabalho de programação pelo tempo ou pelas pessoas. Enquanto se programa uma função, como a que implementa o factorial, concentramo-nos apenas nessa função e ignoramos todos os outros detalhes do programa.

   Num programa com várias funções, cada função pode ser tratada à vez e funções diferentes podem ser desenvolvidas por programadores diferentes, desde que se tomem precauções elementares.

   Ao concentrarmo-nos apenas numa dada função de cada vez, estamos a diminuir a possibilidade de cometer erros.

   Portanto, as boas funções facilitam a implementação e reduzem a ocorrência de erros de programação;

5. Poderíamos substituir uma função por outra que fizesse a mesma coisa, talvez de uma forma mais eficiente, que o programa continuaria a produzir os mesmos resultados.
   5.1. Isto é possível desde que a *interface* da função não se altere. Neste contexto, chamaremos interface ao tipo de retorno, nome e lista de parâmetros da função.
   5.2. Isto é possível por que a função esconde do *cliente*, *i.e.*, do seu utilizador os detalhes da sua implementação, *i.e.*, proporciona abstracção de código;

Se tivéssemos usado funções o nosso programa poderia ser:

```
// Programa que calcula o número de combinações de n elementos agrupados m a m
// Versão 1
import java.util.*;
public class Comb{
	public static void main(String [] args) {
		int n, m, comb;
		Scanner sc = new Scanner(System.in);
		System.out.print("N: ");
		n = sc.nextInt();
		System.out.print("M: ");
		m = sc.nextInt();
		comb = factorial(n) / (factorial (m) * factorial(n-m));
		System.out.print(comb);
	}
	public static int factorial(int n) {
		int f=1;
		for(int i=2; i<=n; i++) f *= i;
		return f;
	}
}
```

Mas as vantagens de utilização de funções não são exclusivas da função factorial. Então porque não usar do mesmo princípio para o próprio cálculo das combinações? Assim, o programa seria:

```
// Programa que calcula o número de combinações de n elementos agrupados m a m
// Versão 2
import java.util.*;
public class Comb {
	public static void main(String [] args) {
		int n, m, comb;
		Scanner sc = new Scanner(System.in);
		System.out.print("N: ");
		n = sc.nextInt();
		System.out.print("M: ");
		m = sc.nextInt();
		comb = combinacoes(n, m);
		System.out.print(comb);
	}
```

```
    public static int combinacoes(int i, int j) {
        return factorial(i) / (factorial (j) * factorial(i-j));
    }
    public static int factorial(int n) {
        int f=1;
        for(int i=2; i<=n; i++) f *= i;
        return f;
    }
}
```

Podemos resumir desde já algumas das vantagens da utilização de funções:

- É um passo em direcção à reutilização de código;
- Aumenta a legibilidade de todo o programa;
- Facilita a manutenção;
- Facilita a implementação;
- Reduz a ocorrência de erros de programação;

Temos portanto aqui um primeiro mecanismo para abstracção de código. A abstracção ou modularidade de funções permite reduzir a complexidade de um problema que possa ser resolvido por uma função principal. Isto é conseguido por decomposição do problema original em problemas mais fáceis resolver com funções mais simples. Como bem nos recordamos de quando falamos da resolução de problemas, a decomposição de um problema em problemas mais simples é uma das nossas principais ferramentas de trabalho.

Definindo funções

Definir uma função significa basicamente atribuir uma instrução composta a um identificador, *i.e.*, ao nome da função. Isto pode ser feito como no exemplo seguinte:

```
public static int factorial(int n) {
    int f=1;
    for(int i=2; i<=n; i++) f *= i;
    return f;
}
```

Isto é:

1º Especifica-se quem poderá usar a função ou, por outras palavras, qual é a sua *visibilidade* ou o seu nível de acesso. Podemos ter quatro níveis diferentes mas para já vamos ver apenas dois desses tipos. No exemplo acima, a palavra-chave public indica que a função pode ser usada por qualquer cliente da classe. Isto é, a função é visível em todo o lado onde a classe for visível. Uma outra possibilidade seria usar a palavra-chave private, o que corresponderia ao nível de acesso privado. O nível de acesso privado significa que a função só pode ser usada por funções da mesma classe. O acesso privado é útil para definir funções auxiliares. As funções auxiliares são funções que servem para construir outras funções mas que por si só não têm interesse para o utilizador.

2º Continuaremos a usar a palavra-chave static e continuaremos a deixar a necessária justificação para mais tarde.

3º Indica-se o tipo de retorno, int no caso deste exemplo. É possível ter funções que não retornam nenhum valor, como é o caso de todas as funções main. Nesse caso, isso deve ser indicado usando a palavra-chave void. Por razões históricas este tipo de função costuma ser chamado *procedimento*, *rotina* ou *sub-programa*.

4º Especifica-se o identificador (ou nome) da função.

5º Indica-se a lista de parâmetros formais bem como os seus tipos. No caso, a função factorial tem apenas um parâmetro, a variável n, do tipo inteiro.

6º Finalmente descreve-se o corpo da função. Chama-se corpo da função a todo o que está entre as chavetas principais.

Parâmetros e tipo de retorno

A variável n que aparece na definição da função factorial chama-se *parâmetro formal* ou simplesmente parâmetro. Os parâmetros formais são os nomes internos dos valores que são passados à função no momento da sua chamada.

O valor da variável i que é usado no momento da chamada à função chama-se *parâmetro real* ou *argumento*. Na altura em que a função é chamada o parâmetro formal recebe uma cópia do valor do parâmetro real (no exemplo, n recebe uma cópia do valor guardado em i). Isto significa que os valores dos

argumentos são primeiro calculados depois os valores encontrados são copiados para os parâmetros formais da função. A este processo dá-se o nome de passagem de argumentos por valor ou por cópia. Assim, a alteração do valor nos parâmetros não se reflecte nos argumentos.

Para retornar um valor, as funções usam a instrução **return**. Uma função pode ter várias instruções **return**. A função termina após a execução do primeiro **return**. O tipo da expressão que se segue ao **return** terá de ser convertível no tipo de retorno da função.

A instrução **return** tem a seguinte sintaxe:

return ***[expressão]*** ;

Chama-se *assinatura* de uma função ao seu nome e à lista de argumentos, com a especificação dos seus tipos e nomes.

Noção de variável local

Sempre que necessário podemos declarar variáveis dentro de uma função, da mesma forma que as podemos declarar dentro de qualquer bloco. No exemplo do factorial, a variável **f** está declarada dentro da função, chamando-se por isso variável local à função.

Consideremos a seguinte função factorial modificada:

```
public static int factorial(int n) {
        int f=1;
        System.out.println(n);
        for(int i=2; i<=n; i++) f *= i;
        return f;
}
```

e o seguinte bloco de código de uma outra função qualquer, ambas da classe **Comb**:

```
{
        int n=1, m=2, fact;
```

```
        System.out.println(n);
        fact = factorial(n+m);
        System.out.println(n);
        System.out.println(fact);
}
```

A execução deste bloco programa produzirá o seguinte resultado:

```
1
3
1
6
```

Vejamos:

1º É declarada a variável n, local ao bloco, e inicializada a 1;

2º É declarada a variável m, local ao bloco, e inicializada com o valor 2;

3º É declarada a variável local ao bloco, fact;

4º É escrito o valor de n, 1, no ecrã;

5º É calculado o valor do parâmetro real da função factorial, obtendo-se o valor 3;

6º O valor 3 é copiado para o argumento da função factorial, n;

7º É iniciada a execução da função factorial

   a. É declarada a variável f, local à função, e inicializada com o valor 1;
   b. É escrito o valor do argumento n, 3, no ecrã;
   c. É executado o ciclo for ;
   d. É retornado o valor actual de f, 6;
   e. A execução da função termina com a libertação da memória onde estiveram guardados os valores da variável local f e do argumento n;

8º O valor 6, retornado pela função, é copiado para a variável fact;

9º É escrito o valor de n, 1, no ecrã;

10º É escrito o valor de fact, 6, no ecrã;

11º A execução do bloco termina com a libertação da memória onde estiveram guardados os valores das variáveis locais n, m, e fact.

Do que acabamos de ver, deve ser claro que quer o argumento quer a variável local da função só existem enquanto a função está em execução. Da mesma

maneira, as variáveis locais ao bloco `n`, `m` e `f` só existem enquanto o bloco estiver em execução.

Deve ser igualmente claro que embora tenham o mesmo nome `n`, a variável local ao bloco e o argumento da função têm tempos de vida e espaços de armazenamento diferentes. É por esta razão que quando o valor da variável local ao bloco `n` é escrito temos 1 e quando o valor do argumento da função é escrito temos 3.

Noção de alcance

O alcance de uma variável é a zona de código em que essa variável está acessível.

- O alcance dos argumentos de uma função é o corpo da função;

- O alcance de variáveis ou declaradas num bloco termina no fim do bloco.

Primeira referência à Qualidade em funções

Para que possamos tirar todo o proveito da utilização de funções temos que construir *boas* funções. Isto é funções me melhorem a *Qualidade* do nosso código.

As características que as boas funções incluem as seguintes:

A Uma função deve ter as propriedades de caixa preta, i.e., para que seja utilizável é suficiente conhecer as entradas e saídas, conhecer o que faz; mas é irrelevante saber como a função está implementada.

   Portanto é muito importante documentarmos devidamente as funções, designadamente fazendo constar uma pequena descrição da tarefa a desempenhar pela função, bem como a lista de argumentos e seu significado, entre outros aspectos relevantes.

B Uma função deve ser fortemente coesa, *i.e.*, deve realizar uma única e bem definida tarefa.

Teremos que voltar a este assunto da Qualidade mais à frente.

Exercícios

1. Escreva e teste uma função que retorne o sinal de um número;
2. Escreva e teste uma função que verifique se um número inteiro é ímpar;
3. Escreva e teste uma função que verifique se um número natural superior a um é ou não primo (um número natural superior a um é primo se só for divisível por 1 e por si próprio).
4. Escreva um programa que apresente os N primeiros números naturais primos. Utilize a função criada no exercício 2.
5. Reescreva o programa que desenha no ecrã um quadrado de caracteres. O caracter e o o lado do quadrado deve ser pedido ao utilizador. Utilize funções. Exemplo de utilização:

```
caracter: z <Enter>
lado: 3 <Enter>
zzz
z z
zzz
```

Primeiras noções de recursividade

Do ponto de vista matemático, uma função diz-se recursiva se for definida em termos dela própria. De igual modo, em programação uma função é recursiva se se chamar a si própria.

Um algoritmo diz-se recursivo quando encontra uma solução para um problema resolvendo uma ou mais instâncias mais simples do problema original. Os algoritmos recursivos são implementados por funções recursivas e as funções recursivas da programação correspondem às definições recursivas das funções matemáticas.

Talvez a função recursiva, mais vezes usada como exemplo em estudos de recursividade seja a função factorial. Como vimos, o factorial de um natural n, designa-se por n!, e é dado por:

$$n! = \begin{cases} n(n-1)\cdots 2.1 & \text{se } n \geq 1 \\ 1 & \text{se } n = 0 \end{cases}$$

Uma alternativa à implementação que fizemos anteriormente para esta função é baseada na observação de que o factorial pode ser definido como:

$$n! = \begin{cases} n(n-1)! & \text{se } n \geq 1 \\ 1 & \text{se } n = 0 \end{cases}$$

A implementação corresponde é:

```
public static int factorial (int n) {
        if (n==0)  return 1;      // ou if ((n==0) || (n==1))
        return n*factorial(n-1);
}
```

Para calcular 3! esta versão recursiva efectua o seguinte encadeamento de operações:

```
factorial(3)
3 * factorial(2)
3 * 2 * factorial(1)
3 * 2 * 1 * factorial(0)
3 * 2 * 1 * 1
3 * 2 * 1
3 * 2
6
```

Podemos expressar sem ciclos qualquer computação que envolva ciclos, usando recursão.

Da mesma forma, é sempre possível transformar a versão recursiva de uma função numa versão iterativa.

A recursividade permite-nos frequentemente expressar algoritmos complexos de uma forma compacta sem sacrificar a eficiência. Mesmo no exemplo simples do factorial, a utilização da recursividade dispensou-nos de usar variáveis locais, simplificando assim a sua implementação.

No entanto, é necessário ter em conta que, quando se usa uma função recursiva, estamos a usar o *stack* [x]do sistema que terá um tamanho proporcional à profundidade de recursão. A profundidade de recursão corresponde ao número máximo de chamadas recursivas da função ao longo da computação (o que tipicamente depende dos dados de entrada). Para certos problemas, o espaço necessário para manter este *stack* pode impedir o uso da solução recursiva.

De forma a podermos garantir, por indução matemática, a correcção das nossas funções recursivas, estas terão de ter duas propriedades básicas:

1. Devem resolver um caso base, que é caracterizado por uma condição de terminação onde é calculado explicitamente um resultado – no caso do factorial corresponde à linha if (n==0) return 1;

2. As chamadas recursivas devem apresentar valores de argumentos cada vez mais pequenos.

Aplicações da recursividade

Pensar de modo recursivo é, em muitos casos, uma ferramenta fundamental para os programadores. Nesta secção, vamos apresentar mais três exemplos que requerem que pensemos de modo recursivo.

Definição sintáctica de instruções

A sintaxe de algumas instruções das linguagens de programação é definida recursivamente. Por exemplo, do que vimos até aqui da linguagem de programação Java podemos dizer que a definição sintáctica de instrução é:

instrução ::= ;
/ ***expressão*** ;
/ { ***{ expressão_lógica }*** }
/ if (***expressão_lógica***) ***instrução***
/ if (***expressão_lógica***) ***instrução*** else ***instrução***
/ switch (***expressão***) {

{ case ***literal*** : ***instrução [*** break;***] }***

*[*default : ***instrução [*** break;***]]*** }

/ break ;
/ while (***expressão_lógica***) ***instrução***
/ do ***instrução*** while (***expressão_lógica***) ;
/ for (***[inicialização]*** ; ***[teste]*** ; ***[actualização]***) ***instrução***
/ continue ;
/ return ***[expressão]*** ;

Isto é, por exemplo a instrução while é definida à custa do próprio conceito de instrução. Uma instrução while pode ser definida com a palavra while seguido de um parêntesis curvo aberto, seguido de uma expressão lógica, seguido de um parêntesis curvo fechado, seguido de uma instrução (a qual pode ser uma nova instrução while) e assim por diante…
Neste caso existem vários casos base que correspondem às instruções mais simples da linguagem, como a instrução nula ou o break.

Conceito de antepassado

Pensemos no conceito de *antepassado*. O meu bisavô ou a minha tetravó são meus antepassados. Um antepassado é alguém que é pai do pai do pai, ou mãe do pai da mãe da mãe, e por aí fora... Aparentemente, trata-se de um conceito de difícil definição. Contudo, se pensarmos de forma recursiva podemos defini-lo facilmente utilizando apenas os conceitos de *pai* e *mãe*, como se segue:

- Se X for pai de Y => X é antepassado de Y.
- Se X for mãe de Y => X é antepassado de Y.
- Se X é pai de Z, e Z é antepassado de Y => X é antepassado de Y.
- Se X é mãe de Z, e Z é antepassado de Y => X é antepassado de Y.

Conjuntos indutivos

As duas formas mais utilizadas de representar conjuntos são: (1) por extensão e (2) por compreensão. O primeiro método trata-se de uma simples enumeração de todos os elementos do conjunto. Por exemplo, o conjunto A pode ser definido em extensão do seguinte modo:

$$A = \{1,2,3,4\}$$

mas também pode ser definido em compreensão como sendo o *Conjunto de números Naturais inferiores a 5*. Formalmente:

$$A = \{x \in N : x < 5\}$$

Os conjuntos também podem ser definidos por indução. Como exemplo, vamos considera a seguinte definição para um certo conjunto C:

- 1 pertence a C
- Se x pertence a C, então x+1 também pertence a C
- nada mais pertence a C

Estas 3 regras definem o conjunto C de maneira indutiva. Trata-se de um conceito que é muito parecido com o conceito de recursividade, visto que estamos a definir C à custa de si próprio. Já agora, de que conjunto é este que estamos a falar?

Exercícios

6. Escreva e teste uma função recursiva que calcule a soma dos primeiros N naturais.
7. Escreva e teste uma função recursiva que escreva uma linha com um dado número de caracteres no ecrã.
8. Baseando na função anterior escreva e teste uma função que desenhe um bloco quadrado de pontos no ecrã, usando um caracter dado. Exemplo de utilização:

```
caracter: x  <Enter>
lado: 3 <Enter>
```

XXX

XXX

XXX

Capítulo

VII

Tabelas

Introduz-se a noção tabela (array).

Existem situações em que necessitamos de guardar muitos valores relacionados uns com os outros. Isso acontece quando queremos guardar o valor do nosso peso ao longo de várias semanas ou mais geralmente quando temos uma colecção de valores e temos de calcular medidas estatísticas. Para usar um exemplo bem nosso conhecido, a média de uma lista de n números, $X_1, X_2, ..., X_n$, é definida como:

$$média = (X_1 + X_2 + ... + X_n) / n$$

e o desvio de cada valor em relação à média é dado pela fórmula:

$$d_i = X_i - média, \text{ para } i = 1,2,...,n$$

Se precisássemos de fazer um programa que calculasse a média de uma lista de 10 números e o desvio de cada número em relação à média, com o que sabemos até agora, teríamos de fazer algo parecido com:

```
{
   float x1, x2, x3, x4, x5, x6, x7, x8, x9, x10;
   float d1, d2, d3, d4, d5, d6, d7, d8, d9, d10;

   // Leitura e validação de x1, ..., x10
   float media = (x1+x2+x3+x4+x5+x6+x7+x8+x9+x10) / 10;
   d1 = x1 - media;
   d2 = x2 - media;
   ...
   d10 = x10 - media;
}
```

A coisa podia complicar-se seriamente se em vez de 10 números fossem necessários guardar 100 ou 1024 valores, o que, diga-se, são necessidades bastante frequentes.

Para situações como esta, a generalidade das linguagens de programação disponibiliza uma estrutura de dados elementar a tabela ou array.

Uma tabela guarda, sobre o mesmo nome, uma colecção de valores do mesmo tipo de dados. Cada um destes valores pode ser acedido por um índice inteiro.

Declaração, definição e acesso a elementos de uma tabela

Declara-se um inteiro i escrevendo:

```
int i;
```

declara-se uma tabela de inteiros a escrevendo:

```
int [] a;
```

Ao contrário da declaração do inteiro i, que reserva espaço em memória para conter um valor inteiro, a declaração da tabela a não reserva espaço para os seus elementos. Para reservar espaço para os elementos utiliza-se o operador new.

```
int [] a = new int[5];
```

A instrução anterior reserva espaço para uma tabela de cinco inteiros.

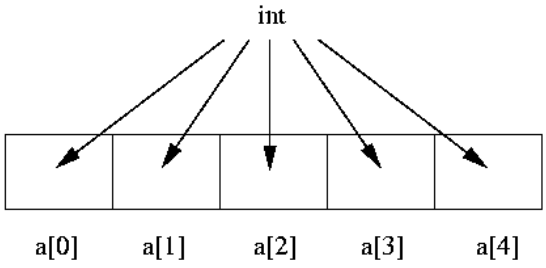
Em linguagens como o Java, o C ou C++, o primeiro elemento de uma tabela encontra-se no índice (ou posição) 0. Por isso, a instrução anterior define as variáveis:

a[0], a[1], a[2], a[3] e a[4]

Depois de definida a tabela, os seus elementos podem ser acedidos e modificados individualmente, tal e qual como nas variáveis que vimos até agora. Por exemplo:

```
a[0] = 54;
int i = a[3];
```

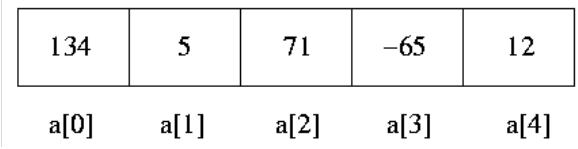
Podemos pensar na tabela a como uma sequência de 5 posições de memória consecutivas, todas do tipo int, indexadas pelos índices 0,1, 2, 3 e 4.



depois de efectuar as seguintes instruções:

```
a[0] = 134;
a[1] = 5;
a[2] = 71;
a[3] = -65;
a[4] = 12;
```

o estado da tabela a passaria a ser



Podemos verificar facilmente isso, escrevendo

```
for(int i=0; i <a.length ; i++)
        System.out.println("a["+i+"]: "+a[i]);
```

onde, como facilmente se percebe, a.length devolve o comprimento ou o tamanho da tabela a.

Em alternativa às instruções de inicialização anterior, poderíamos ter usado a seguinte instrução:

```
int [] a = {134, 5, 71, -65, 12};
```

sendo que à parte direita desta instrução de inicialização abreviada, chamaremos *literal* de tabela.

Não é demais, termos uma palavra de precaução relativamente à utilização de tabelas. O Java deixa para o utilizador a responsabilidade de usar índices não negativos e menores de que o tamanho da tabela. Se tentarmos aceder a posições que não existem na tabela, por estarem fora dos seus limites; por exemplo, se tentássemos aceder a a[12] o compilador não daria qualquer erro; No entanto teríamos um erro em tempo de execução e o programa terminaria.

Podemos ter tabelas de qualquer tipo de dados. No caso geral, a definição de uma tabela é feita da seguinte forma:

tipo [] ***identificador*** = new ***tipo*** [***expressão_algébrica***] ;

Depois desta breve introdução, estamos em condições de fazer um programa para calcular a média de uma colecção de N valores e seus respectivos desvios em relação à média.

```
import java.util.*;
public class Estatistica {
  public static void main (String [] args) {
     int n;
    Scanner sc = new Scanner(System.in);
     do{
        System.out.print("N: ");
        n = sc.nextIn();
        if (n<=0)  System.out.println("Número inválido.");
     }
     while(n<=0);
     float soma=0;
     float [] x = new float[n];

     // leitura dos valores e cálculo da média
     int i;
      for(i=0; i<n; i++ ) {
           System.out.print("x"+i+"? ");
           x[i] = sc.nextFloat();
```

```
            soma = soma + x[i];
        }
        float media = soma / n;

      // cálculo dos desvios
      float [] desvio = new float[n];
      for(i=0; i<n; i++ )
        desvio[i] = x[i] - media;

      // Escrita  da média, dos números, e dos respectivos desvios
      System.out.println("A média: " + media );
      for(i=0; i<n; i++ )
         System.out.println("x"+i+"= " + x[i] + " desvio["+i+"]: "+desvio[i]);
    }
}
```

Noção de referência

Não é possível fazer directamente a atribuição entre duas tabelas. Isto é, o código:

```
int [] a = new int[5];
a[0] = 134;
a[1] = 5;
a[2] = 71;
a[3] = -65;
a[4] = 12;
int [] b = a;
```

Não faz certamente o que estamos à espera. O que se observa é que se modificarmos o i-nésimo elemento da tabela b isso reflecte-se no i-nésimo elemento da tabela a, e vice-versa. Isto é, se escrevermos:

```
b[0] = 0;
System.out.print(a[0]);
```

verificamos que a saída é 0 e não 134, como estaríamos possivelmente à espera. Isto acontece por que quer o identificador a quer o identificador b referenciam a mesma posição de memória. Por outras palavras, a e b são ambos referências para o primeiro elemento da mesma tabela. Uma referência guarda o endereço (ou o valor de) uma posição da memória. Em Java esse valor não pode ser manipulado directamente pelo programador. Isto é, embora

se saiba que a contem o endereço do primeiro elemento da tabela a, não é possível aceder ao elemento i da tabela escrevendo a+i.

Uma vez que a e b são referências, para copiarmos uma tabela, teremos que fazer uma atribuição elemento a elemento, como se segue.

```
int [] a = new int[5];
a[0] = 134;
a[1] = 5;
a[2] = 71;
a[3] = -65;
a[4] = 12;
int [] b = new int[a.length];
for( int i=0; i<a.length; i++)    b[i]=a[i];
```

Tabelas como argumento de funções

As tabelas podem ser passadas como argumentos de funções. Para exemplificar vamos escrever uma função para a classe Estatistica que pede n valores ao utilizador, calcula e devolve a sua média:

```
// Versão 1.1
import java.util.*;
public class Estatistica {
        public static void main (String [] args) {
            int n;
            Scanner sc = new Scanner(System.in);
            do{
              System.out.print("N: ");
              sc = nextInt();
              if (n<=0)  System.out.println("Número inválido.");
            }
            while(n<=0);
            float [] x = new float[n];
            float media = leTabelaFloat(x);

            // cálculo dos desvios
            int i;
            float [] desvio = new float[n];
            for(i=0; i<n; i++ )
                desvio[i] = x[i] - media;
            // Escrita  da média, dos números, e dos respectivos desvios
            System.out.println("A média: " + media );
           for(i=0; i<n; i++ )
```

```
            System.out.println("x"+i+"= " + x[i] + " desvio["+i+"]: "+desvio[i]);
        }

        private static float leTabelaFloat(float [] tf) {
            // Le os valores da tabela passada como argumento e calcula a sua média
            float soma = 0.0f;
            Scanner sc = new Scanner (System.in);
            for(int i=0; i< tf.length; i++ ) {
                System.out.print("x"+i+"? ");
                tf[i] = sc.nextFloat();
                soma += tf[i];
            }
            return (soma / tf.length);
        }
}
```

Algumas notas sobre esta versão 1.1 da classe Estatística:

1. Uma vez que, tal como está definida, a função leTabelaFloat parece só ter interesse para esta classe, foi declarada com o nível de acesso privado. Assim, um cliente desta classe não poderá usar esta função; esse previlégio está reservado apenas para as funções da própria classe;

2. A declaração do parâmetro formal tf, já nos é familiar; corresponde à declaração de uma tabela de valores do tipo float;

   Note-se que esta declaração é em tudo semelhante à declaração do parâmetro formal da função main. Isto significa que a função main quando invocada recebe como argumento uma tabela cujos valores são do tipo String. Deixaremos para depois o estudo do tipo String. No entanto, podemos fazer desde já uma experiência interessante. Após compilar a classe

```
public class ArgMain{
public static void main(String [] args) {
        for(int i=0; i<arg.length; i++)
                System.out.println(args[i]);
    }
 }
```

   execute-se o programa escrevendo:

```
java ArgMain 1 dois 3.5
```

o que obtemos?

Desvendamos pois, mais um dos mistérios da enigmática expressão

```
public static void main(String [] args)
```

3. A chamada à função faz-se como se a tabela x fosse uma variável como outra qualquer, i.e., basta escrever leTabelaFloat(x). No entanto, os elementos de x não são copiados para os elementos de tf; apenas o valor da referência x é copiado para tf.

4. Também teria sido possível definir a tabela a ler dentro da função leTabelaFloat. Neste caso, a versão seria mais ineficiente:

```
// Versão 1.2
    import java.util.*;
    public class Estatistica {
       public static void main (String [] args) {
          int n;
          Scanner sc = new Scanner(System.in);
          do{
            System.out.print("N: ");
            n = sc.nextInt();
            if (n<=0)  System.out.println("Número inválido.");
          }
          while(n<=0);
          float [] x = leTabelaFloat(n);

          // calculo da média e dos desvios
          int i;
          float soma=0.0f;
          for(i=0; i<n; i++ ) soma +=x[i];
          float media = soma /n;
          float [] desvio = new float[n];
          for(i=0; i<n; i++ )  desvio[i] = x[i] - media;

          // Escrita da média, dos números, e dos respectivos desvios
          System.out.println("A média: " + media );
          for(i=0; i<n; i++ )
            System.out.println("x"+i+"= " + x[i] + " desvio["+i+"]: "+desvio[i]);
```

```
    }

    private static float [] leTabelaFloat(int n) {
        // Cria uma tabela de n floats, onde guarda os valores pedidos ao
        // utilizador ; Retorna a tabela preenchida
        Scanner sc = new Scanner(System.in);
        float [] tf  = new float [n];
        for(int i=0; i< tf.length; i++ ) {
                System.out.print("x"+i+"? ");
                tf[i] = sc.nextFloat();
        }
        return tf;
    }
}
```

Exercícios

1. Escreva um programa que peça ao utilizador um número natural n, crie uma tabela com n elementos, preencha-a com números aleatórios e finalmente apresente os elementos da tabela ao utilizador. Valide a entrada do utilizador e use funções.

2. Altere o programa anterior de forma a que este apresente também o maior e o menor dos números aleatórios.
3. Reescreva usando tabelas o programa que pede um número natural no sistema de numeração decimal e converte-o para binário.
4. Reescreva usando tabelas o programa que lê um inteiro e que determine a frequência (*i.e.*, o número de vezes) de cada um dos seus diferentes algarismos .
5. Escreva um programa que peça ao utilizador um número natural n e apresente uma tabela de conversão de decimal para binário, com n elementos. Exemplo de utilização:

```
N? 4 <Enter>
0      0
1      1
2     01
3     11
4    100
```

Notas e referências

[i] Existem algumas excepções. Por exemplo, no Smalltalk um inteiro poderá ser tão grande quanto a capacidade da máquina o permitir. Nesta linguagem, só se utiliza um número fixo de bits para inteiros pequenos. A partir de um certo valor a representação dos inteiros deixa de ser fixa e passa a utilizar tanta memória quanta a necessária.

[ii] Ver http://www.unicode.org

[iii] Naturalmente que para que este procedimento de três passos funcione é necessário que o JDK – Java Development Kit esteja instalado. O JDK é um conjunto de ferramentas para desenvolvimento de programas em Java, de uso livre, existe para os vários sistemas operativos e pode ser descarregado a partir de http://java.sun.com

[iv] Teremos que voltar aqui mais tarde já que um ficheiro pode ter vários elementos, incluindo a definição de várias classes. No entanto, apenas uma dessas classes será pública. O ficheiro terá sempre a extensão **java** e um nome igual ao nome da classe pública.

[v] Teremos oportunidade de verificar isso em detalhe quando implementarmos operadores de auto-atribuição para os nossos próprios tipos de dados.

[vi] A palavra **switch** pode ser traduzida por comutador.

[vii] Por não fazer sentido, variáveis do tipo **boolean** não podem ser usadas na instrução **switch**.

Notas e referências

[viii] Em bom rigor, em vez do ciclo deveríamos usar a fórmula da soma da progressão aritmética de razão 1, *i.e.*, 1 + 2 + ... + N = (1+N) * N/2

[ix] Dado que estamos a exercitar os ciclos não vale usar a expressão 1 + (int) Math.log10(n).

[x] Teremos oportunidade de ver em detalhe mais à frente o que se entende por stack.